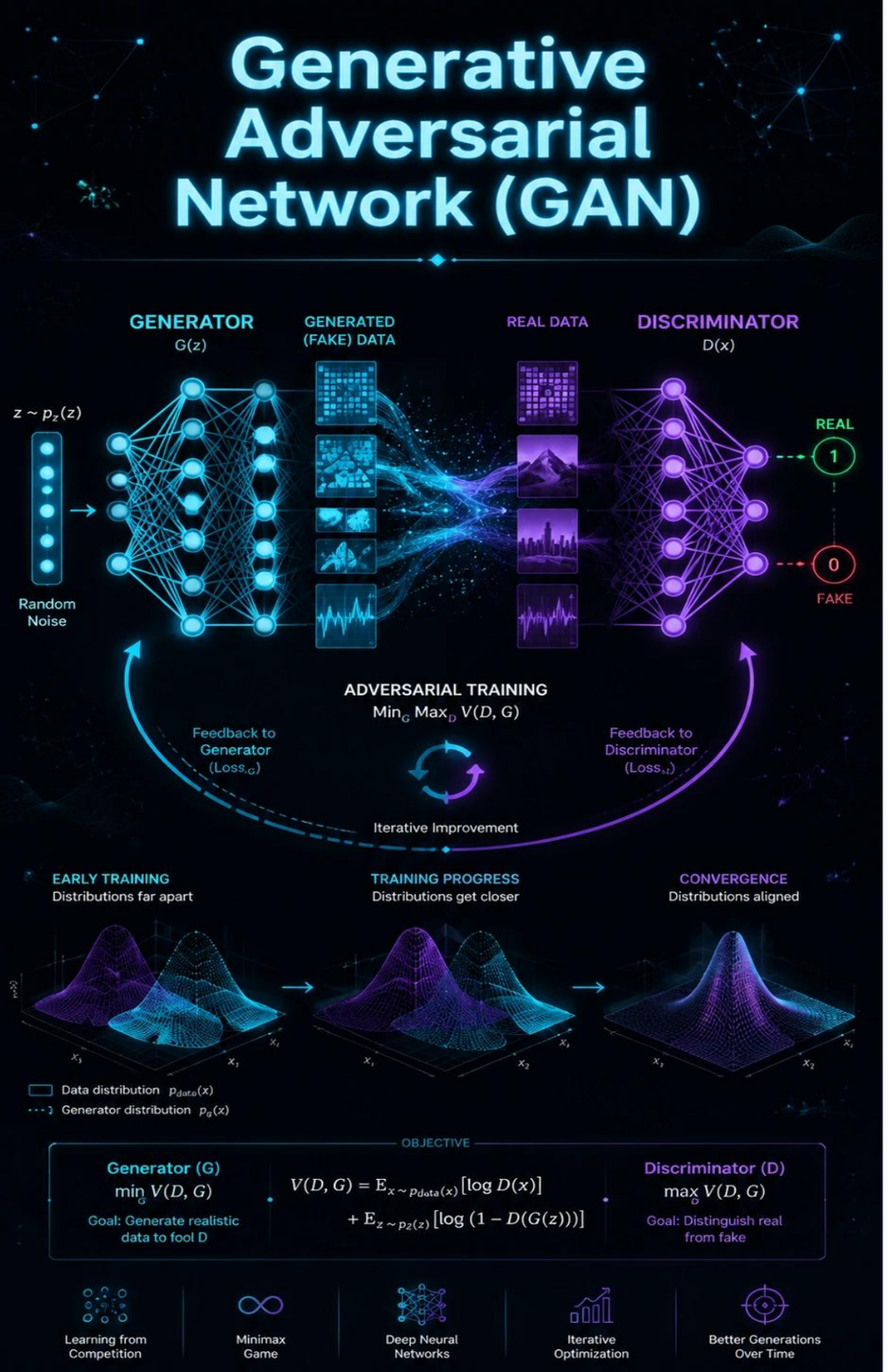


Generative Adversarial Network (GAN)

A Complete Lecture on Generative Adversarial Network

Undergraduate Engineering — Neural Networks & Deep Learning

Introduction

Introductory guide to Generative Adversarial Networks (GANs) and their promise!
Faizan Shaikh, June 15 , 2017

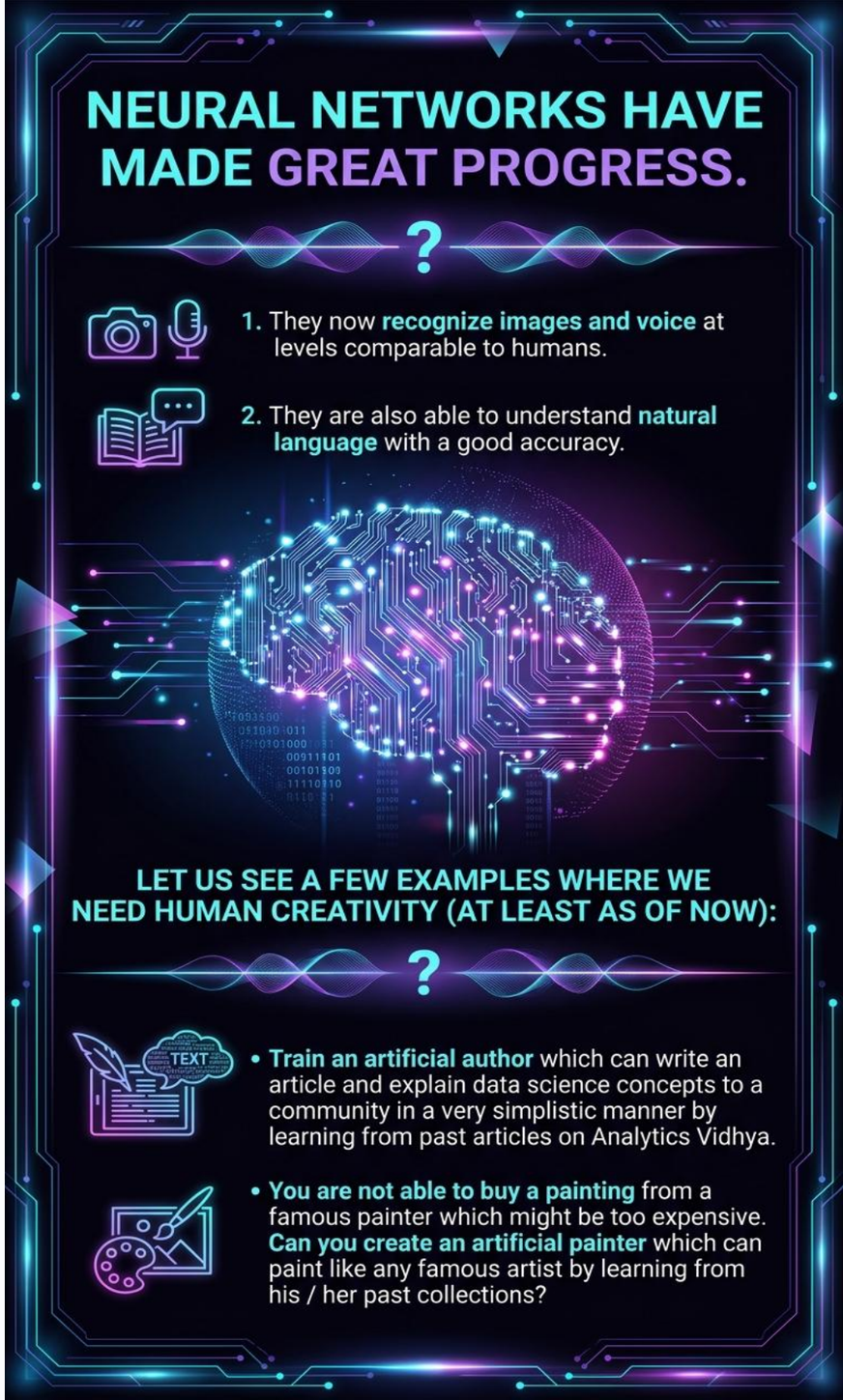
Neural Networks have made great progress.

1. They now recognize images and voice at levels comparable to humans.
2. They are also able to understand natural language with a good accuracy.

Let us see a few examples where we need human creativity (at least as of now):

• Train an artificial author which can write an article and explain data science concepts to a community in a very simplistic manner by learning from past articles on Analytics Vidhya.

• You are not able to buy a painting from a famous painter which might be too expensive. Can you create an artificial painter which can paint like any famous artist by learning from his / her past collections?



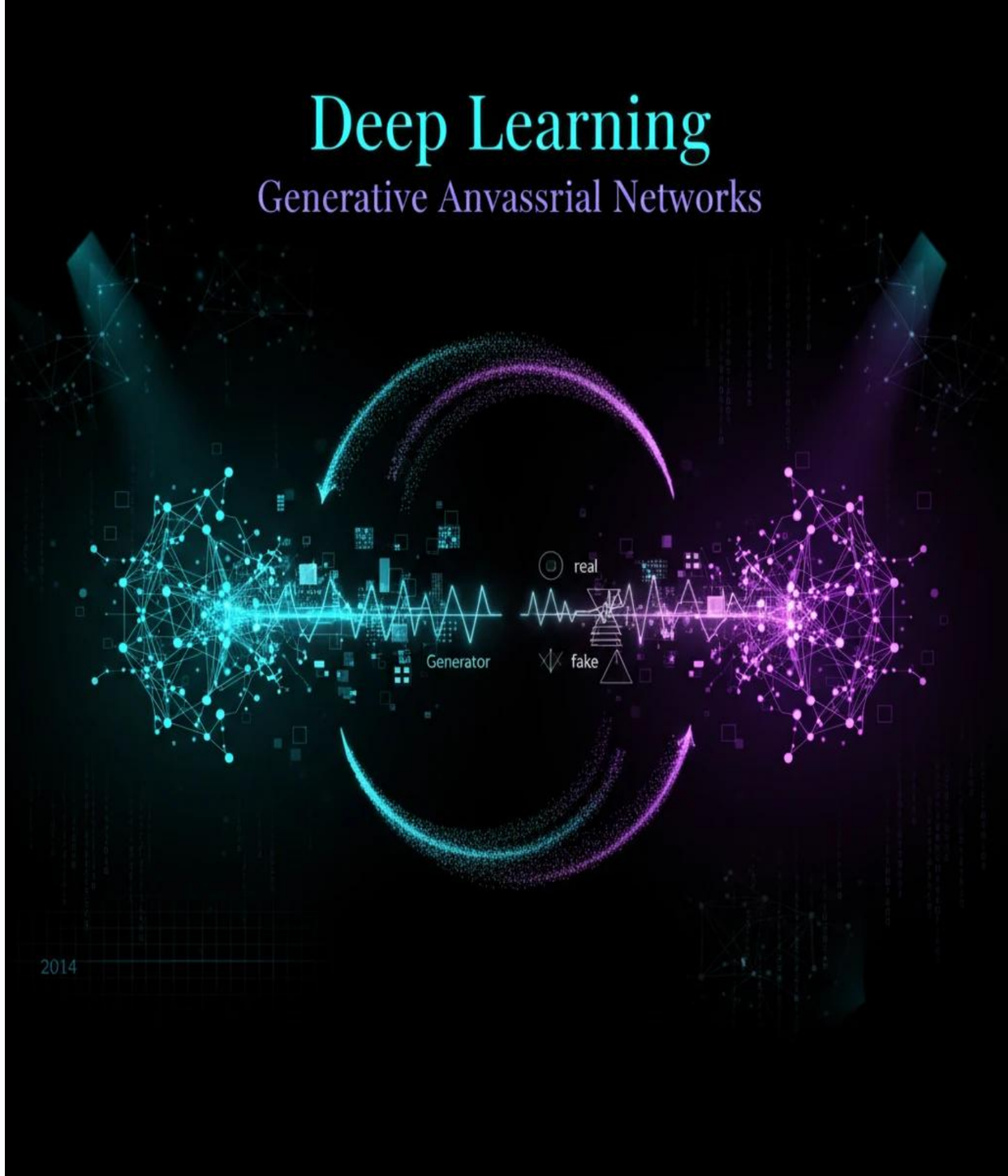
GANs: A mechanism to generate the desired patterns

It seems we should develop a mechanism in order to generate fake patterns which are so similar to real patterns.

Ian Goodfellow (in 2014) introduce GANs for such purposes.

- Yann LeCun , a prominent figure in Deep Learning Domain said in his Quora session that:

“(GANs), and the variations that are now being proposed is the most interesting idea in the last 10 years in ML, in my opinion.”

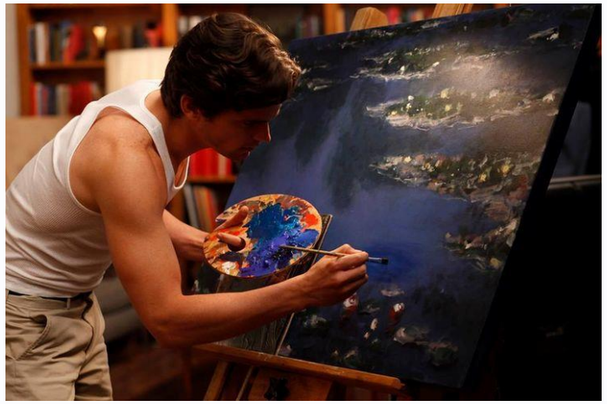


But what is a GAN?

Let us take an analogy to explain the concept:

If you want to get better at something, say **chess**; what would you do? You would compete with an opponent better than you. Then you would analyze what you did wrong, what he / she did right, and think on what could you do to beat him / her in the next game.

You would repeat this step until you defeat the opponent. This concept can be incorporated to build better models. So simply, for getting a powerful hero (viz generator), we need a more powerful opponent (viz discriminator)!



Another analogy from real life

A slightly more real analogy can be considered as a relation between forger and an investigator.

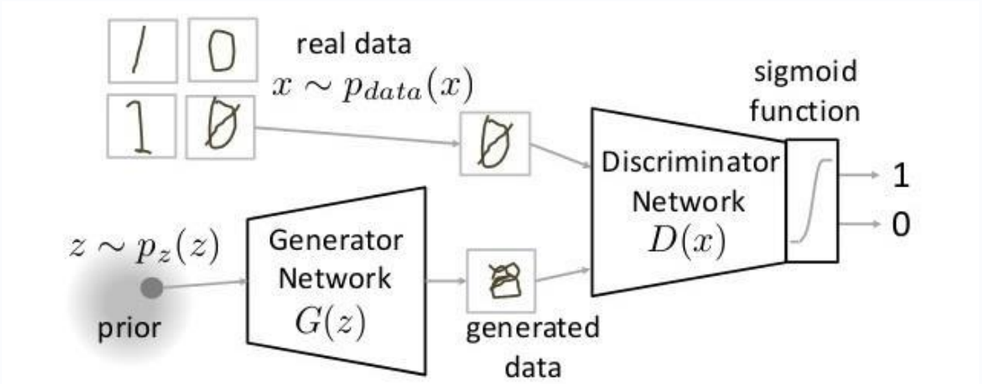
The task of a forger is to create fraudulent imitations of original **paintings** by famous artists. If this created piece can pass as the original one, the forger gets a lot of money in exchange of the piece.

On the other hand, an art investigators task is to catch these forgers who create the fraudulent pieces. How does he do it? He knows what are the properties which sets the original artist apart and what kind of painting he should have created. He evaluates this knowledge with the piece in hand to check if it is real or not.

This contest of forger vs investigator goes on, which ultimately makes world class investigators (and unfortunately world class forger); a battle between good and evil.

How do GANs work?

As we saw, there are two main components of a GAN-Generator Neural Network and Discriminator Neural Network.



1. The Generator Network takes a random input and tries to generate a sample of data.
- In the above image, we can see that generator $G(z)$ takes an input z from $p(z)$, where z is a sample from probability distribution $p(z)$.
2. It then generates a data which is then fed into a discriminator network $D(x)$
3. The task of Discriminator Network is to take input either from the real data or from the generator and try to predict whether the input is real or generated. It takes an input x from $p_{data}(x)$ where $p_{data}(x)$ is our real data distribution. $D(x)$ then solves a binary classification problem using sigmoid function giving output in the range 0 to 1.
- Let us define the notations we will be using to formalize our GAN,
- $P_{data}(x) \rightarrow$ the distribution of real data
  $X \rightarrow$ sample from $p_{data}(x)$
  $P(z) \rightarrow$ distribution of generator
  $Z \rightarrow$ sample from $p(z)$
  $G(z) \rightarrow$ Generator Network
  $D(x) \rightarrow$ Discriminator Network

Generative Adversarial Networks

Now the training of GAN is done (as we saw above) as a
“fight between generator and discriminator.”

This can be represented mathematically as:

2. The First Part: $\mathbb{E}_{x \sim p_{data}(x)}[\log D(x)]$
   - **What it represents:** The "best-case scenario" where **real data** (x) is given to the discriminator.
   - **The Goal:** The discriminator wants to correctly identify real data as 100% real. Therefore, the **discriminator tries to maximize this term to 1.**

The Second Part: $\mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))]$

What it represents: The "worst-case scenario" where random noise (z) is turned into a **fake sample** by the generator ($G(z)$) and handed to the discriminator.

The Tug-of-War:

- **The Discriminator (D)** wants to catch the fake. It wants $D(G(z))$ to be 0 (fake). If $D(G(z)) = 0$, then $\log(1 - 0) = \log(1) = 0$. So, the **discriminator tries to maximize this entire term to 0** (bringing it closer to zero from negative numbers).
- **The Generator (G)** wants to fool the discriminator. It wants $D(G(z))$ to be 1 (real). If it succeeds, $\log(1 - 1) = \log(0) = -\infty$. This **minimizes** the entire function, which is exactly the generator's goal.

$$\min_G \max_D V(D.G)$$

$$V(D,G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}\left[\log\left(1 - D\left(G(z)\right)\right)\right]$$

In our function $V(D,G)$ the first term is entropy that the data from real distribution ($p_{data}(x)$) passes through the discriminator (also known as (aka) best-case scenario).

The discriminator tries to maximize this to 1.
The second term is entropy that the data from random input ($p(z)$) passes through the generator, which then generates a fake sample which is then passed through the discriminator to identify the fakeness (aka worst-case scenario).

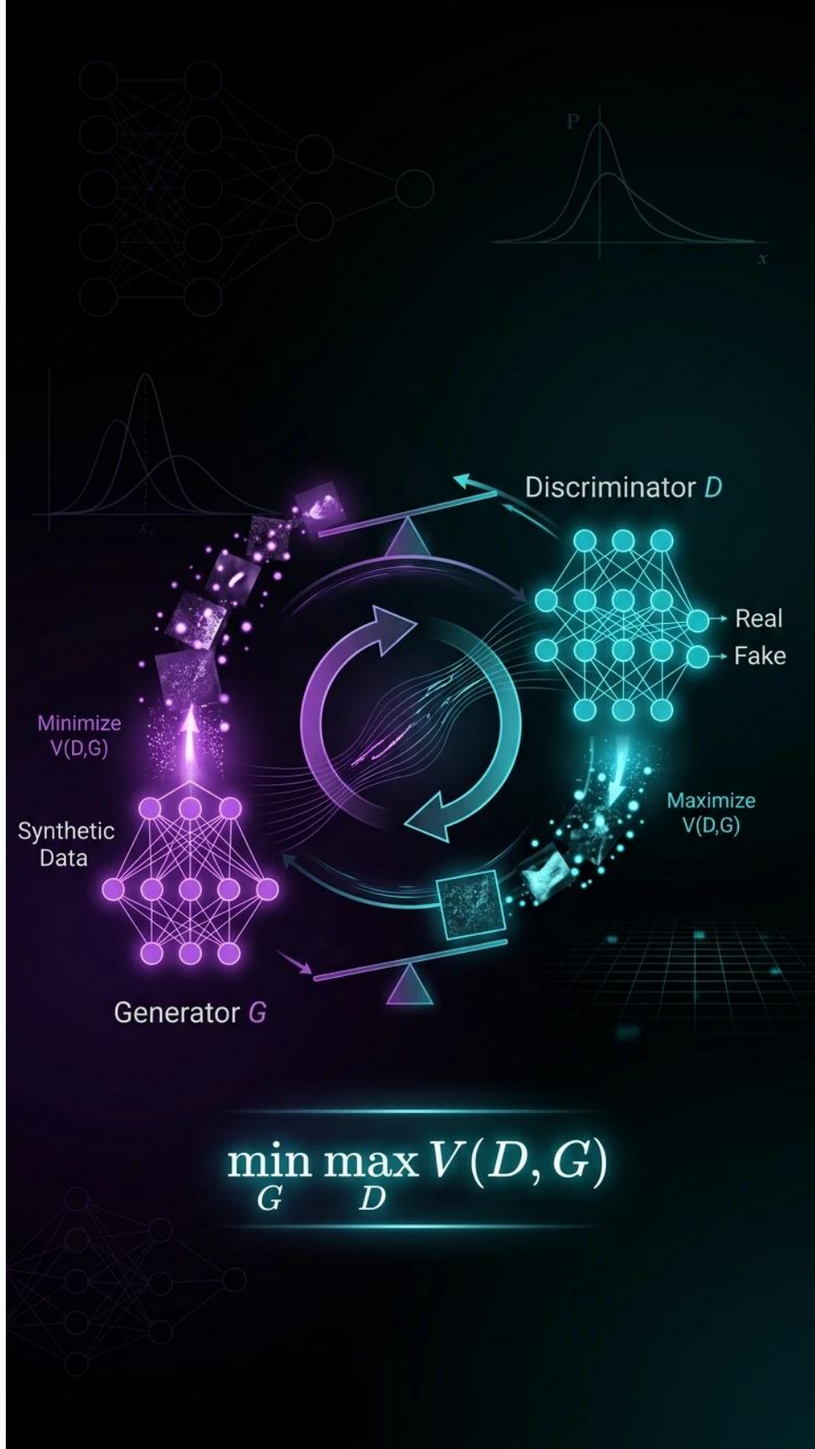
In this term, discriminator tries to maximize it to 0
(i.e. the log probability that the data from generated is fake is equal to 0).
of training a GAN is taken from game theory called the minimax game.

Why GANs Use a Minimax Objective (Game-Theoretic View)

GAN training is fundamentally a two-player zero-sum game.
Each player optimizes an objective directly opposed to the other:
Discriminator D: wants to *maximize* the value function(becoming better at telling real from fake).
Generator G: wants to *minimize* the same value function(making fake samples look real).
This structure corresponds exactly to the **minimax game** in game theory:

$$\min_G \max_D V(D,G)$$

In a zero-sum game, one player's gain is the other player's loss, which is why the GAN objective naturally forms a minimax formulation.



Nash Equilibrium in GANs

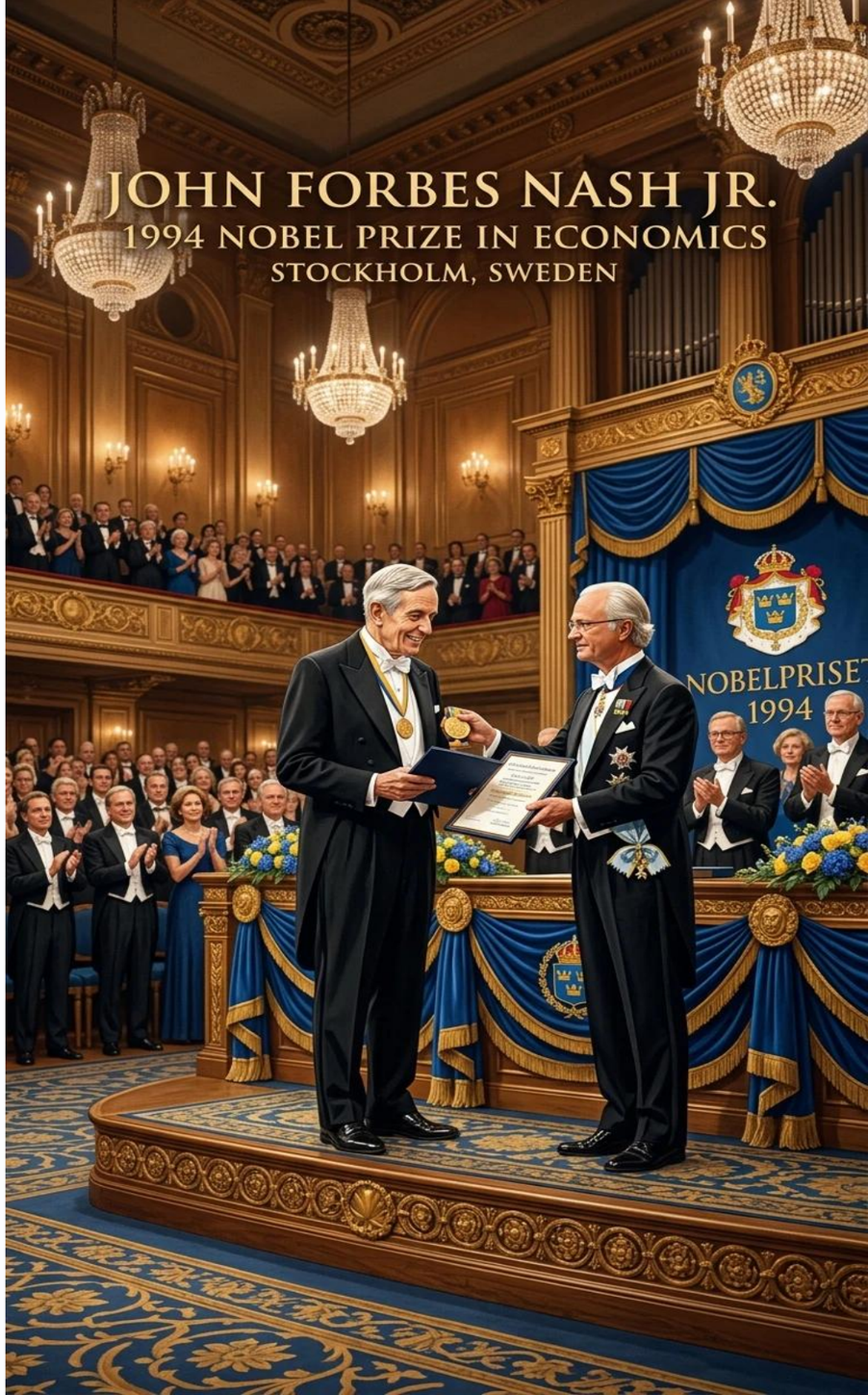
From **game theory**, the ideal training outcome is a **Nash equilibrium—a point** where:

- The **discriminator** cannot improve because the **generator** produces perfect fakes.
- The generator cannot improve because the discriminator is guessing with probability $\frac{1}{2}$. At equilibrium:

$$p_g(x) = p_{data}(x)$$

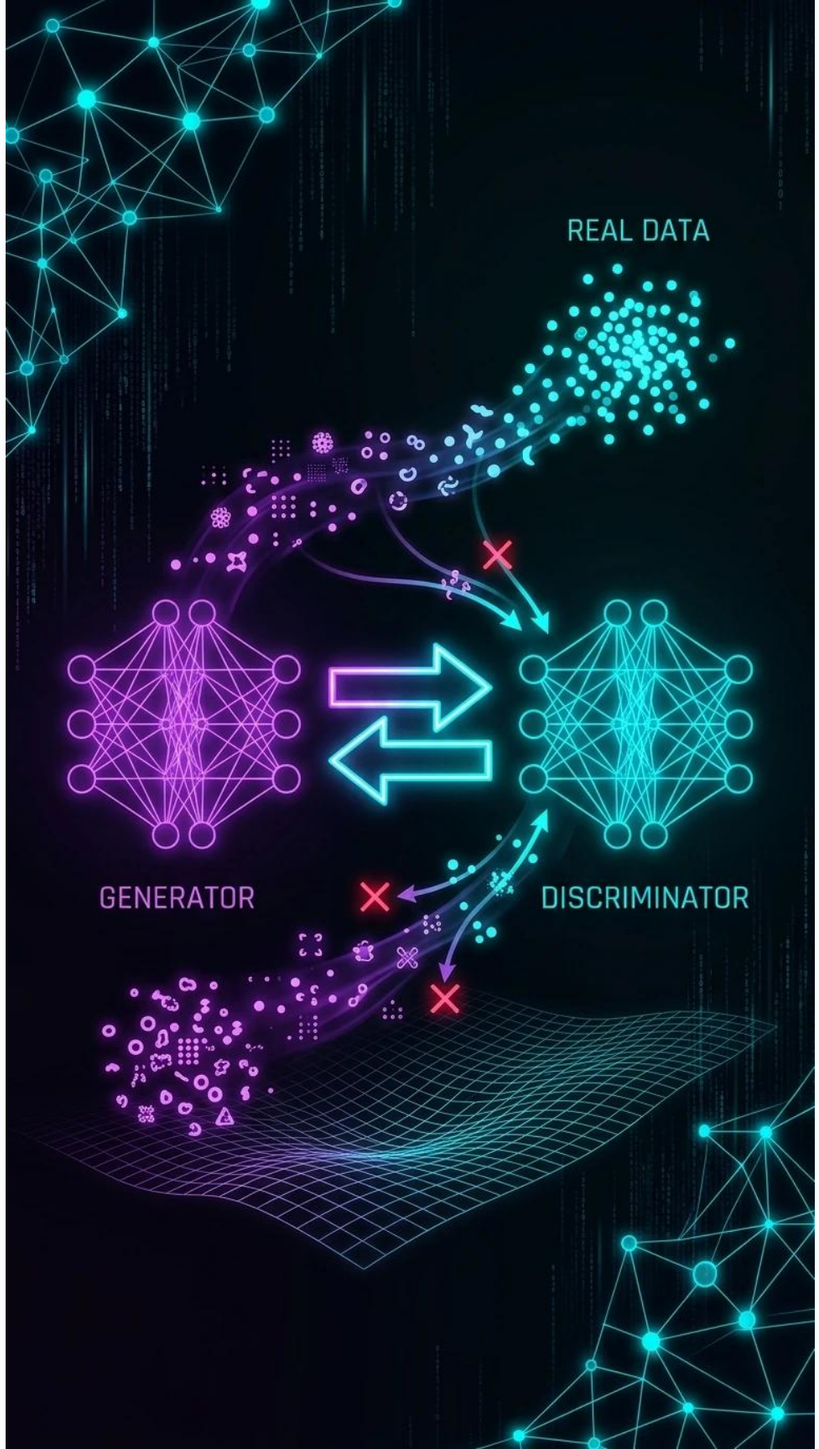
$$D(x) = 0.5 \text{ for all } x$$

This means the **generator's distribution becomes indistinguishable from real data**, and **the discriminator is maximally uncertain**.



Intuition Behind the Zero-Sum Game Dynamics

GAN dynamics **mirror** actual **competition**:
The **Generator** tries to **fool** the **discriminator** by producing increasingly **realistic samples**.
The **Discriminator** tries to **catch** the **generator** by becoming better at **identifying fakes**.
This constant push-and-pull produces:
Adversarial learning, where **both networks improve through competition**.
Dynamic optimization, where **the loss landscape shifts as one player learns**.
Emergent realism, because the **generator must approximate the true data distribution to "win"**.
GANs work because competition forces both networks into a state where the generator approximates the real data extremely well.



Generative Adversarial Networks

So overall, the discriminator is trying to maximize our function V.

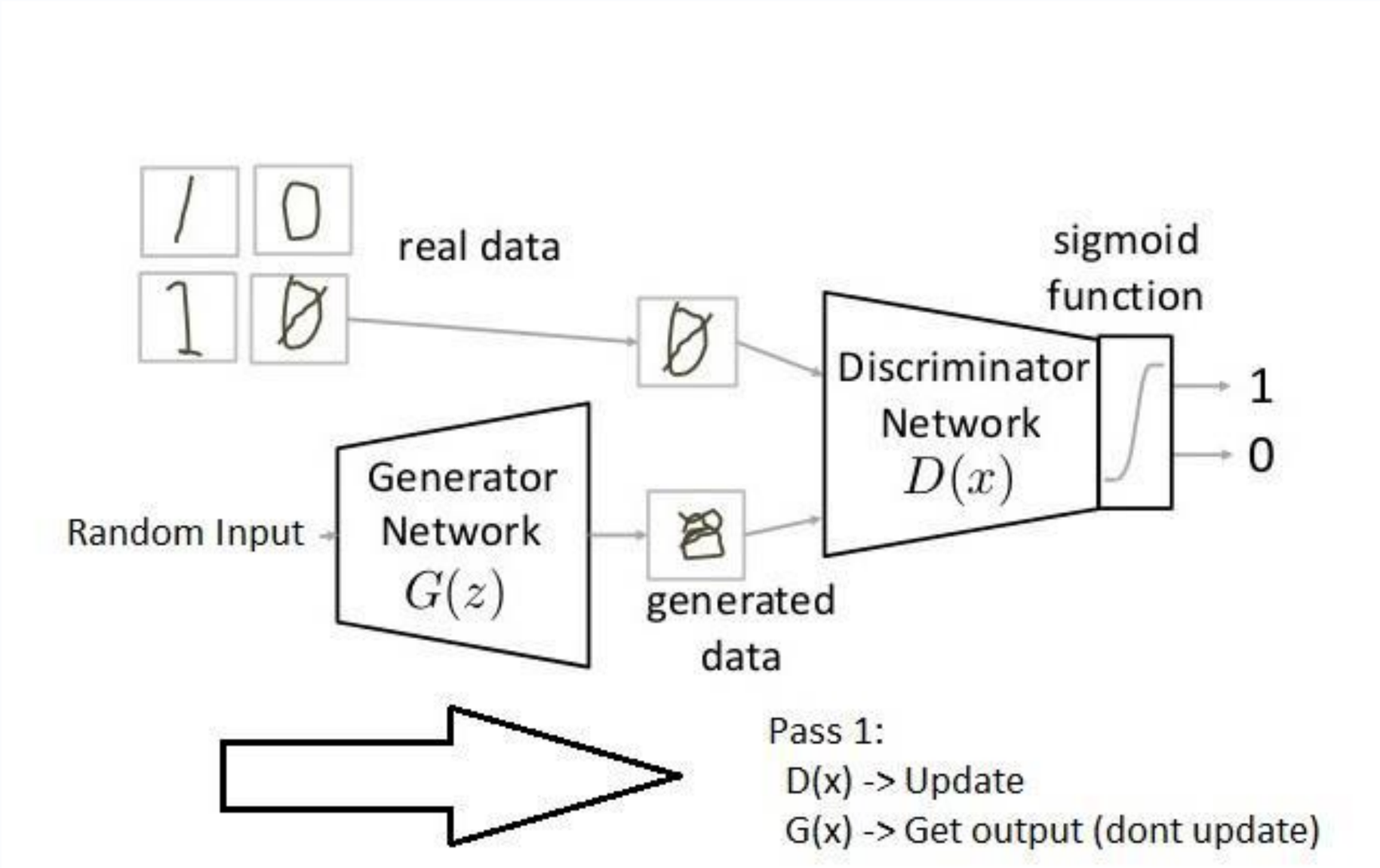
On the other hand, **the task of generator is exactly opposite, i.e. it tries to minimize the function V** so that the differentiation between real and fake data is bare minimum.

This, in other words is a cat and mouse game between generator and discriminator!

Note: This method of training a GAN is taken from game theory called the minimax game.

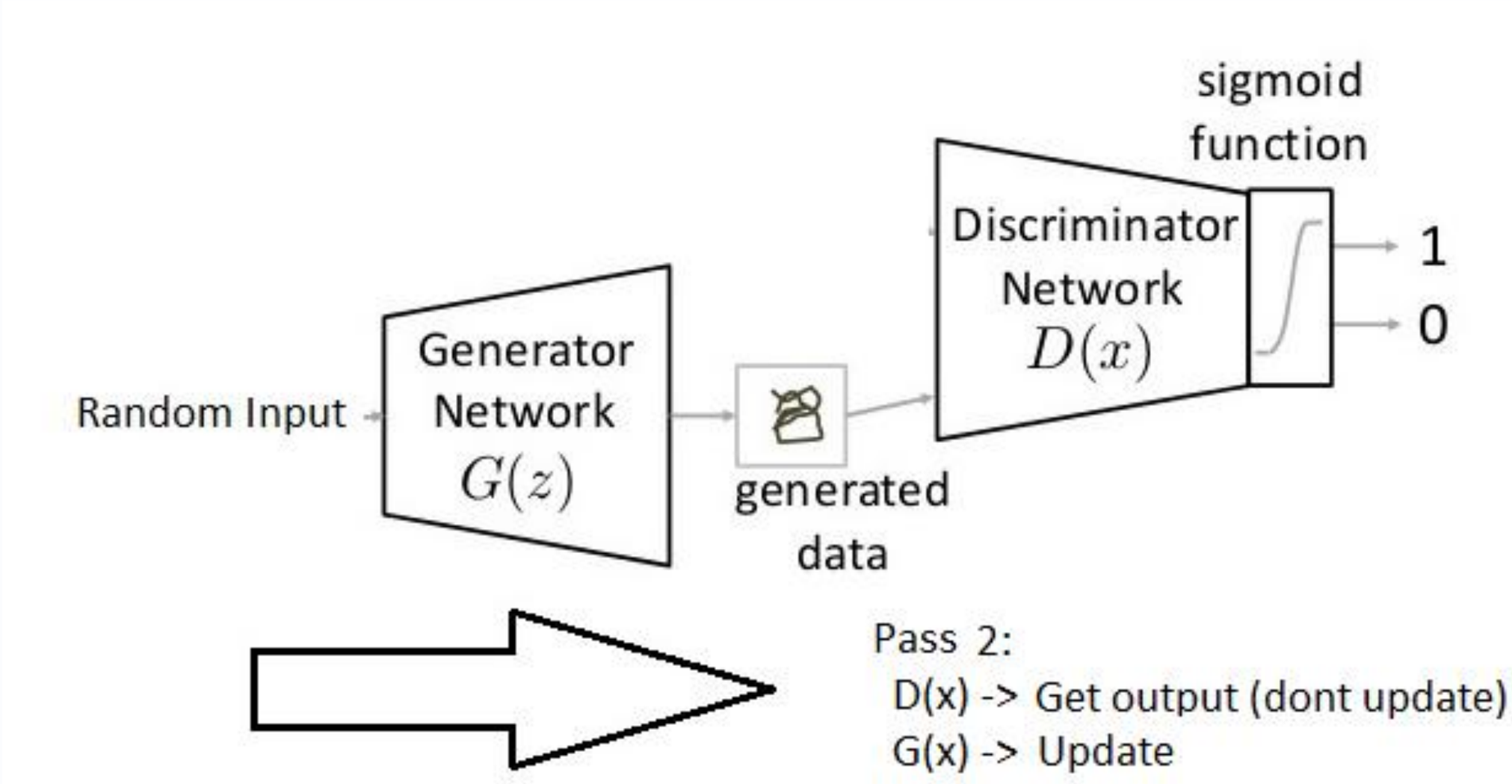
Generative Adversarial Networks

Pass 1: Train discriminator and freeze generator (freezing means setting training as false. The network does only forward pass and no backpropagation is applied).



Generative Adversarial Networks

Pass 2: Train generator and freeze discriminator.



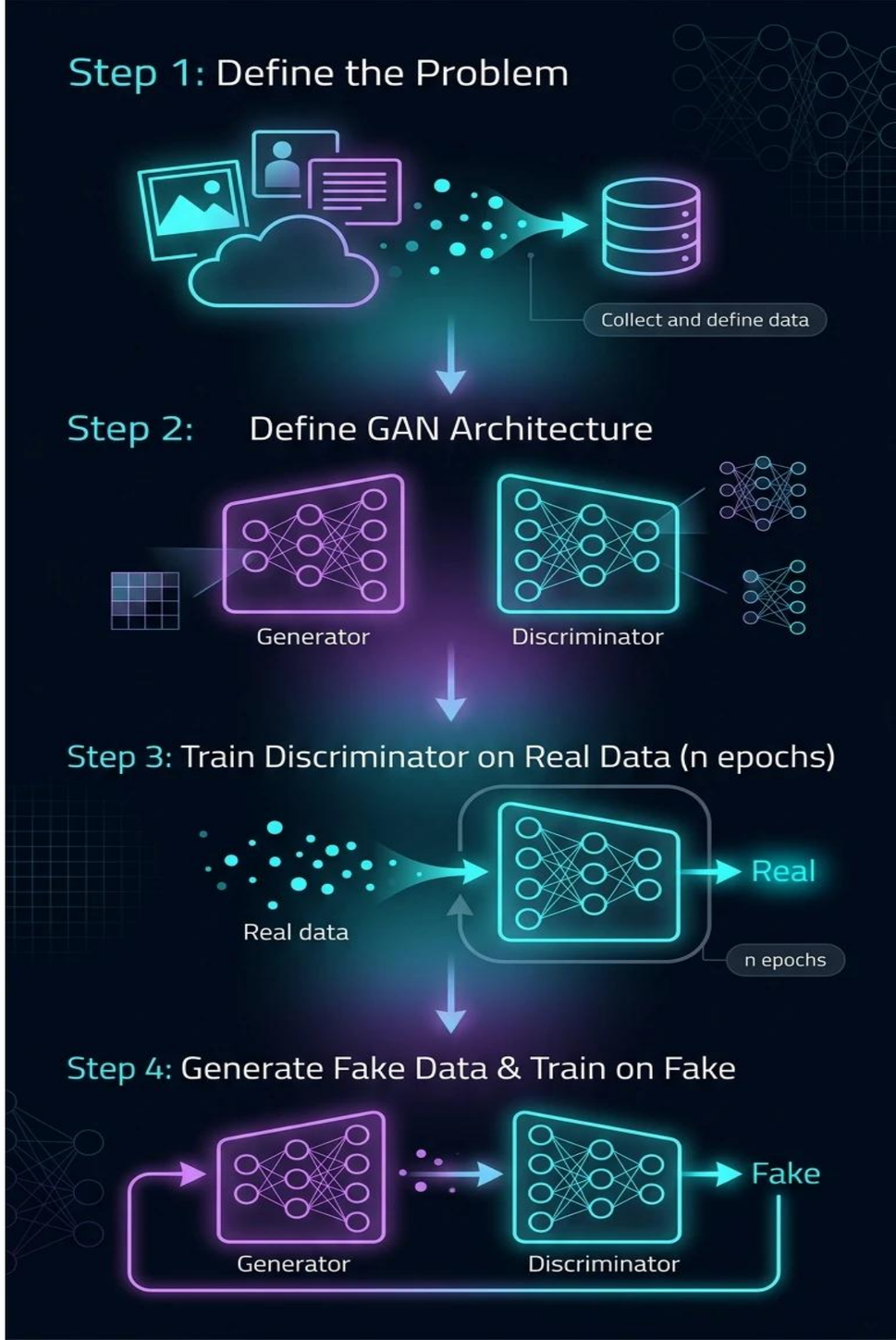
Steps to train a GAN

- **Step 1: Define the problem.** Do you want to generate fake images or fake text. Here you should completely define the problem and collect data for it.

- **Step 2: Define architecture of GAN.** Define how your GAN should look like. Should both your generator and discriminator be multi layer perceptrons , or convolutional neural networks? This step will depend on what problem you are trying to solve.

- **Step 3: Train Discriminator on real data for n epochs.** Get the data you want to generate fake on and train the discriminator to correctly predict them as real. Here value n can be any natural number between 1 and infinity.

- **Step 4: Generate fake inputs for generator and train discriminator on fake data.** Get generated data and let the discriminator correctly predict them as fake.

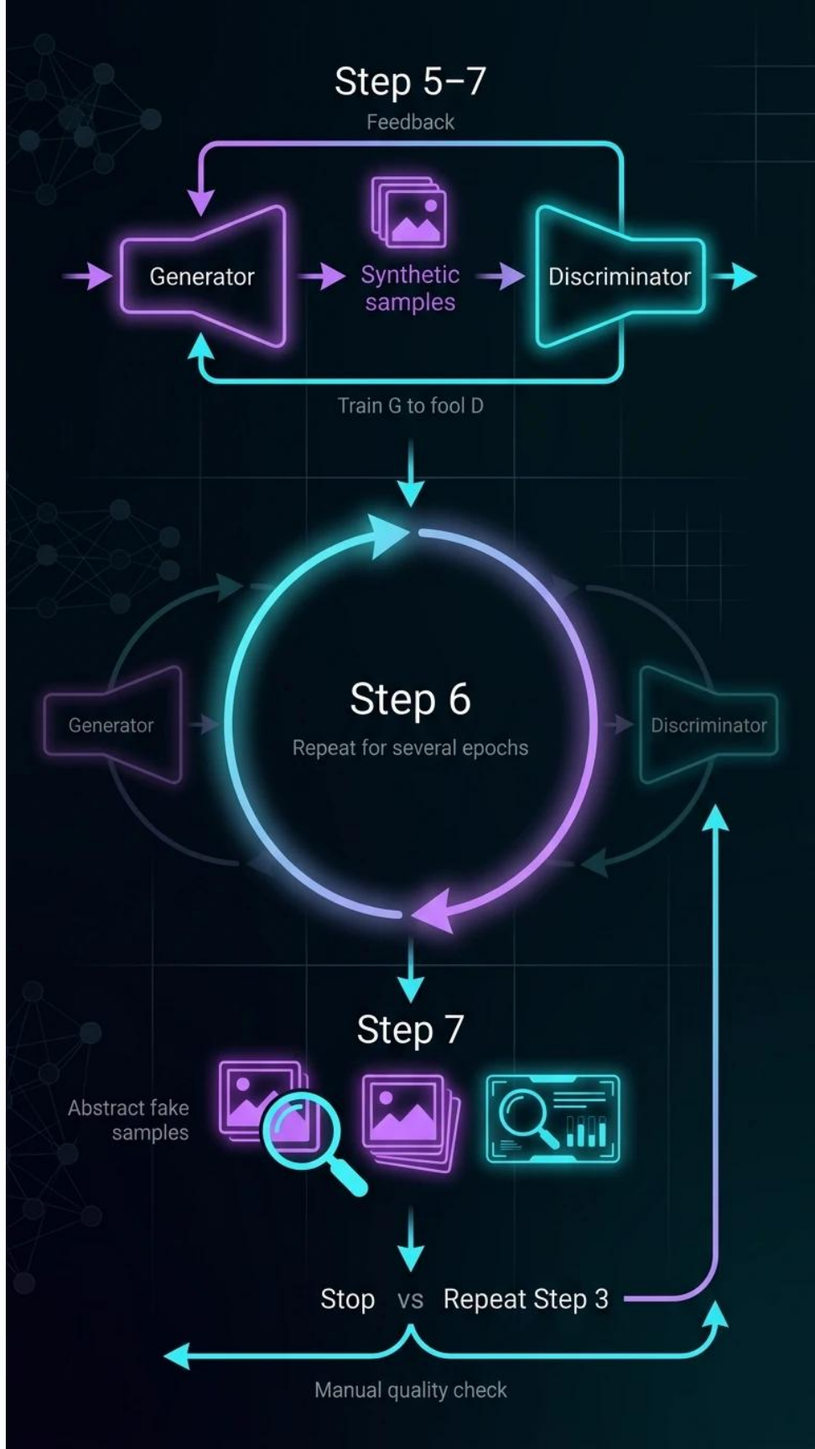


Steps to train a GAN

- **Step 5: Train generator with the output of discriminator. Now when the discriminator is trained, you can get its predictions and use it as an objective for training the generator. Train the generator to fool the discriminator.**

- **Step 6: Repeat step 3 to step 5 for a few epochs.**

- **Step 7: Check if the fake data manually if it seems legit. If it seems appropriate, stop training, else go to step 3. This is a bit of a manual task, as hand evaluating the data is the best way to check the fakeness. When this step is over, you can evaluate whether the GAN is performing well enough.**



Generative Adversarial Networks

A pseudocode of GAN training can be thought ou

Gradient Ascent	Add the gradient to go up	**Discriminator:** Maximize its accuracy in telling real from fake.
Gradient Descent	Subtract the gradient to go down	**Generator:** Minimize the gap between its fakes and real data (fool the D).

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Sample minibatch of m examples $\{x^{(1)}, \dots, x^{(m)}\}$ from data generating distribution $p_{data}(x)$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{\mathrm{m}} \sum_{i=1}^{m} \left[logD\left(x^{(i)}\right) + \log\left(1 - D\left(G\left(z^{(i)}\right)\right)\right)\right].$$

The Discriminator uses gradient ascent because its objective function represents its overall accuracy at spotting both real and fake data. It wants to maximize this score, climbing as close to 100% accuracy as possible.

Train Discriminator

end for

- Sample minibatch of m noise samples $\{z^{(1)}, \dots, z^{(m)}\}$ from noise prior $p_g(z)$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{\mathrm{m}} \sum_{i=1}^{m} \log\left(1 - D\left(G\left(z^{(i)}\right)\right)\right).$$

The Generator uses gradient descent because it wants to minimize the Discriminator's ability to catch it. It wants to drive the error rate of its deception down to zero.

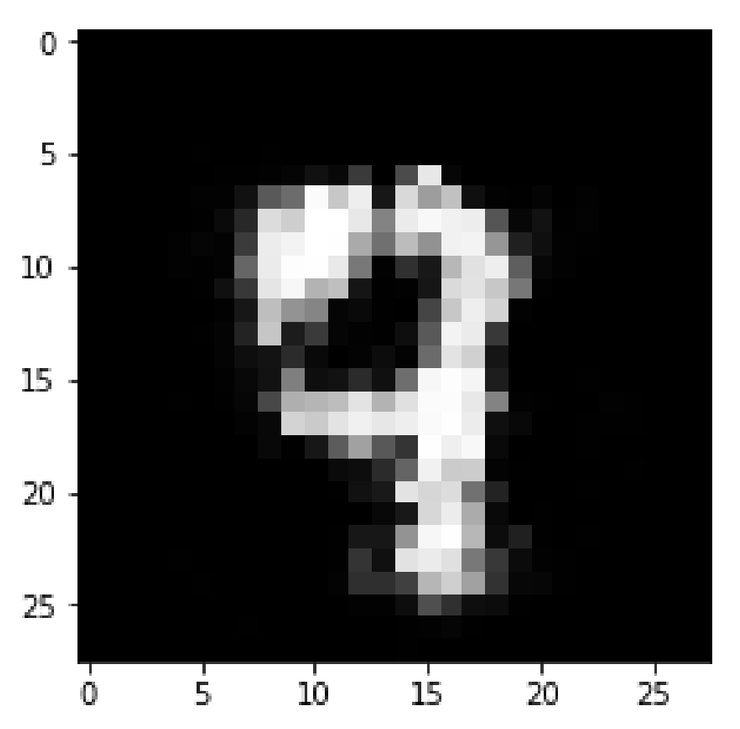
Train Generator

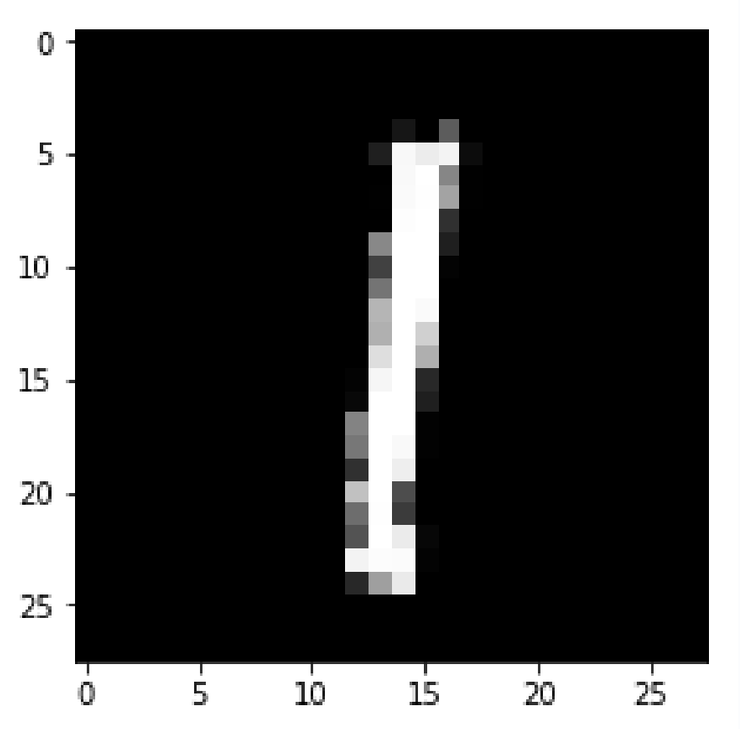
end for

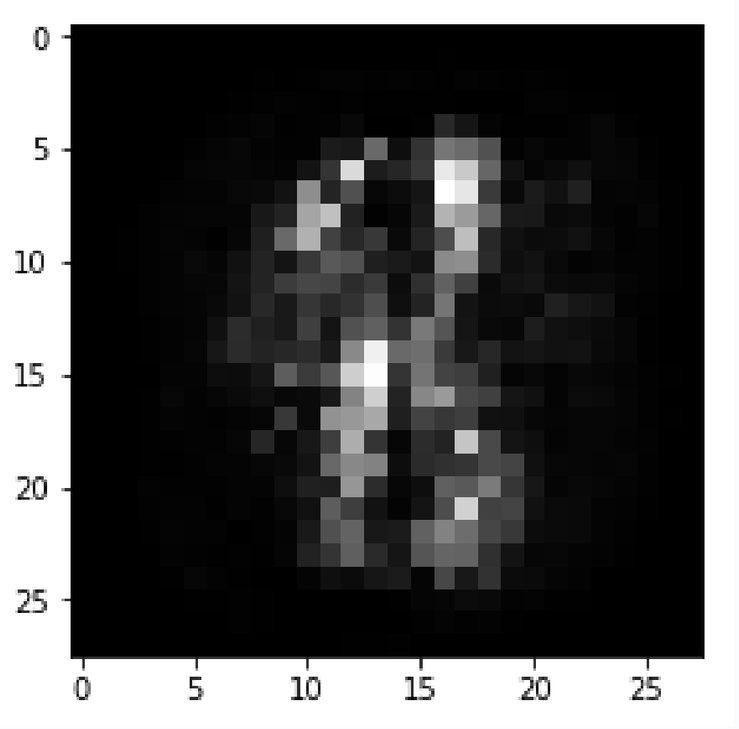
This changes the Generator's parameters just enough to make its next batch of fake data slightly more convincing, lowering the chance that the Discriminator will catch the trick.

Generative Adversarial Networks

We will try to generate digits by training a GAN.
After training for 100 epochs, I got the following generated images.

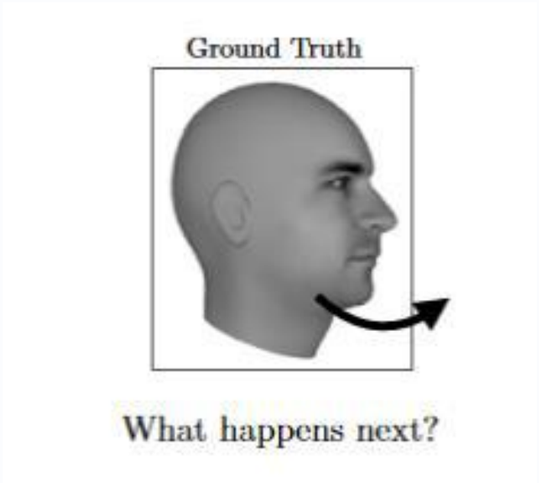


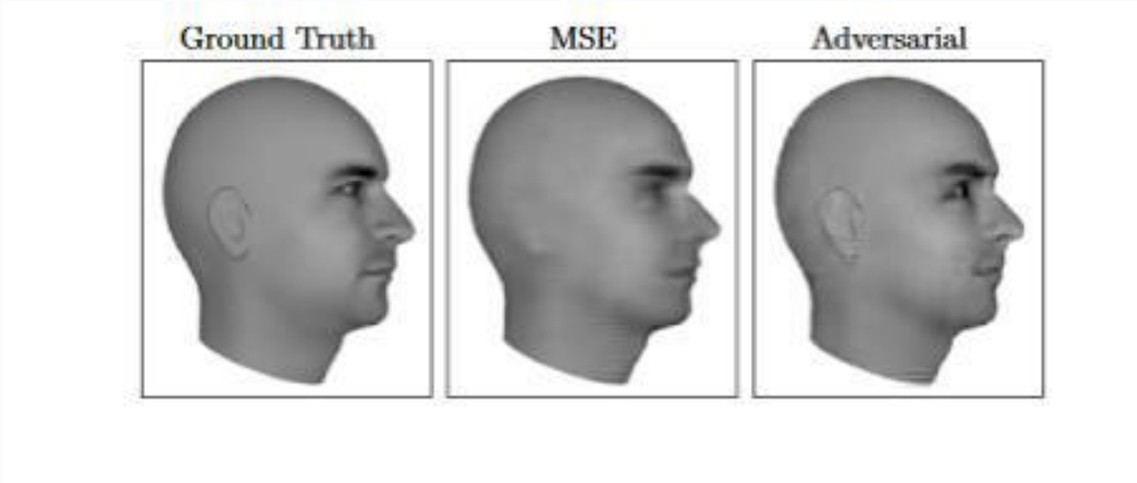




Applications of GAN

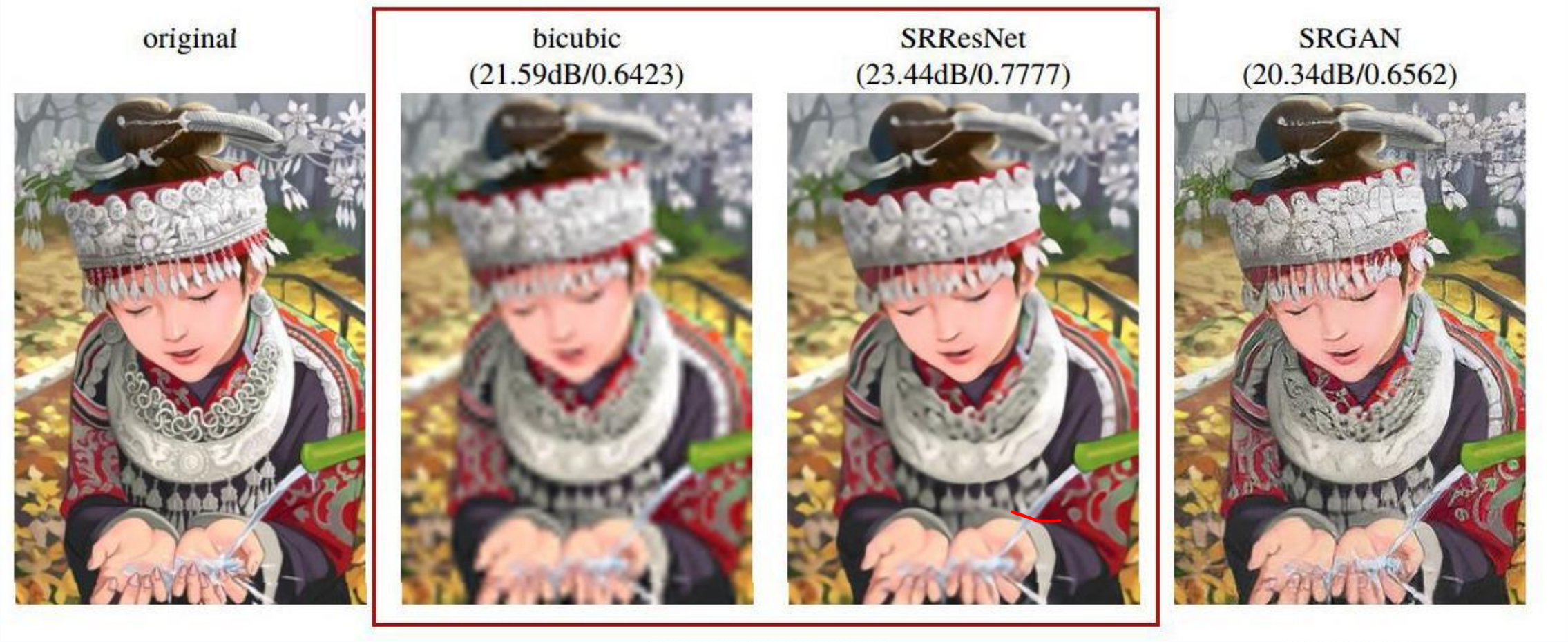
Predicting the next frame in a video: You train a GAN on video sequences and let it predict what would occur next.





Applications of GAN

Increasing Resolution of an image: Generate a high-resolution photo from a comparatively low resolution.



Applications of GAN

Image to Image Translation: Generate an image from another image. For example, given on the left, you have labels of a street scene and you can generate a real looking photo with GAN. On the right, you give a simple drawing of a handbag and you get a real looking drawing of a handbag.

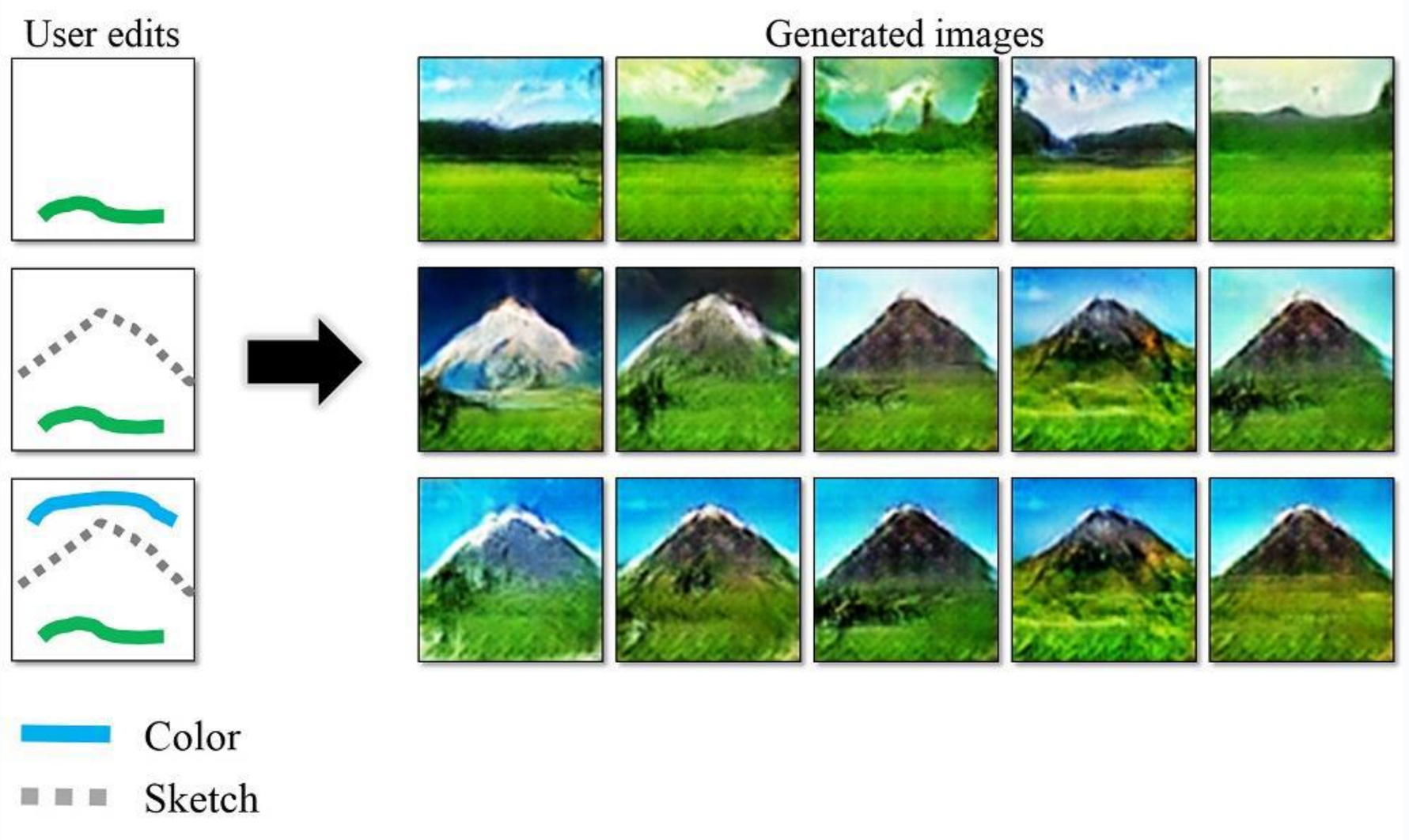
Applications of GAN

Text to Image Generation: Just say to your GAN what you want to see and get a realistic photo of it.

This bird has a yellow belly and tarsus, grey back, wings, and brown throat, nape with a black face

Applications of GAN

Interactive Image Generation (IGAN): Draw simple strokes and let the GAN draw an impressive picture for you.



General Plan

A **General Plan** to generate **solutions** for challenging **problems.**

Image 1: The Standard GAN Training Loop (The Math)

This image establishes the baseline rules for training a regular GAN using mini-batches of data:

1. **Discriminator Update (k steps):** * It takes a batch of random noise vectors (z) and a batch of real data (x).
   - It climbs up its gradient (**gradient ascent**) to maximize its ability to call real data "1" and fake data "0".
2. **Generator Update (1 step):** * It creates new fake samples from fresh noise (z).
   - It slides down its gradient (**gradient descent**) to minimize the expression $\log(1 - D(G(z)))$. This is mathematically equivalent to maximizing the chance that the Discriminator is fooled.

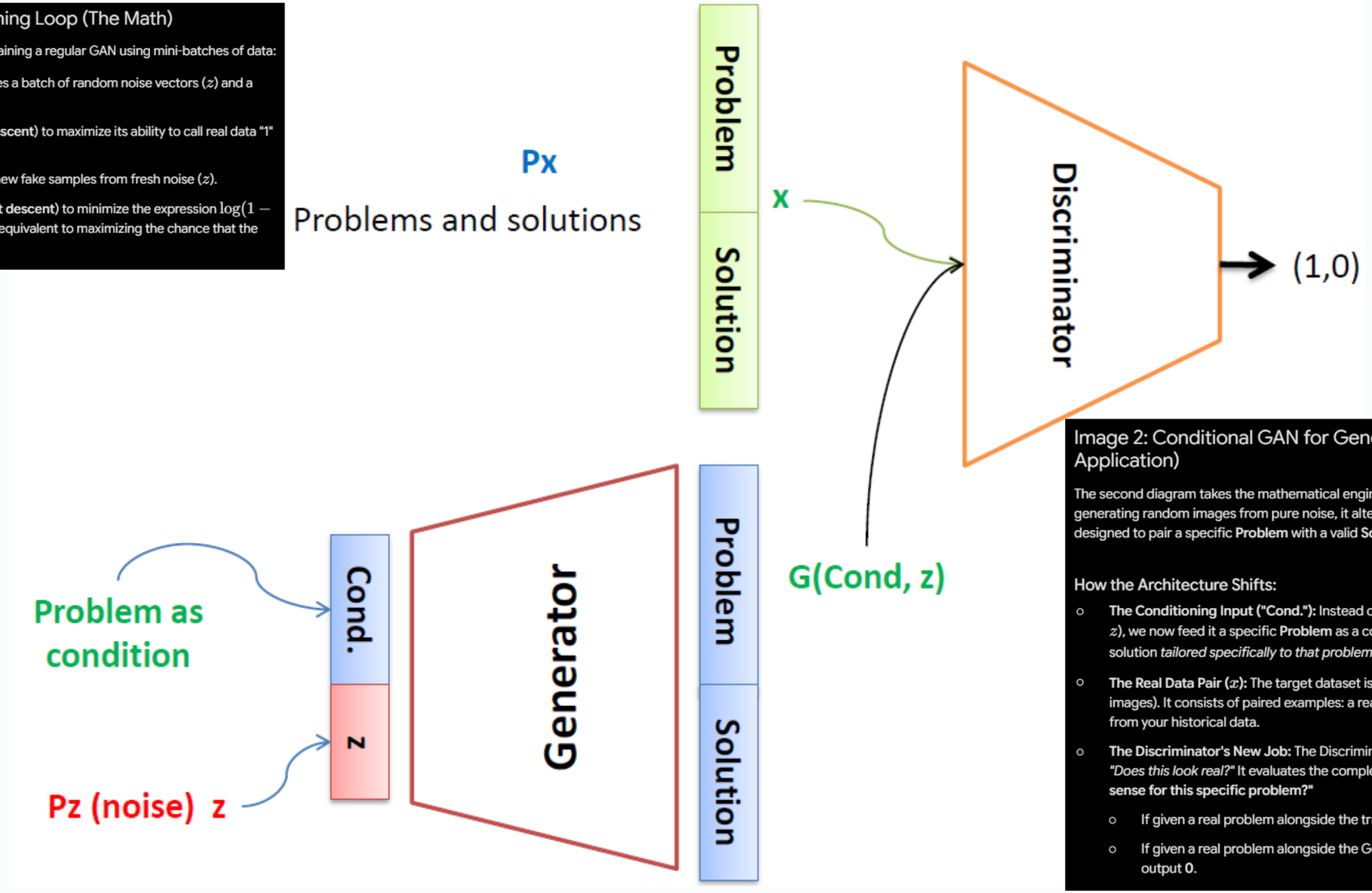


Image 2: Conditional GAN for Generating Solutions (The Application)

The second diagram takes the mathematical engine from Image 1 and modifies it. Instead of generating random images from pure noise, it alters the architecture into a **Conditional GAN** designed to pair a specific **Problem** with a valid **Solution**.

How the Architecture Shifts:

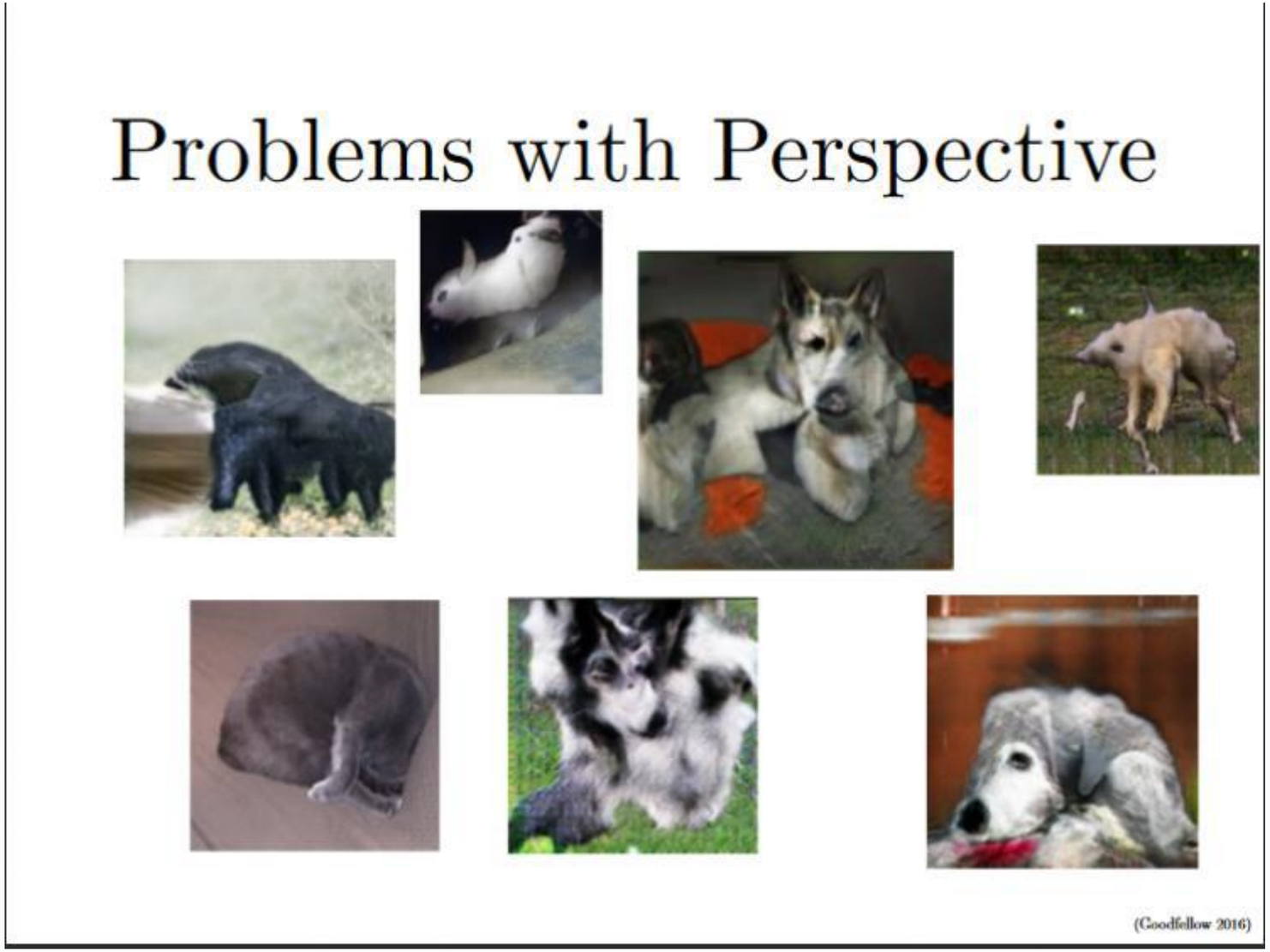
- **The Conditioning Input ("Cond."):** Instead of feeding the Generator *only* random noise (z), we now feed it a specific **Problem** as a condition. The Generator must now create a solution *tailored specifically to that problem*, denoted as $G(\text{Cond.}, z)$.
- **The Real Data Pair (x):** The target dataset is no longer single items (like standalone images). It consists of paired examples: a real **[Problem + True Solution]** combination from your historical data.
- **The Discriminator's New Job:** The Discriminator doesn't just look at a solution and ask, *"Does this look real?"* It evaluates the complete pair. It asks: **"Does this solution make sense for this specific problem?"**
  - If given a real problem alongside the true solution, it outputs **1**.
  - If given a real problem alongside the Generator's synthetic solution, it tries to output **0**.

Challenges With GAN

Problem with Counting: GANs fail to differentiate how many of a particular object should occur at a location. As we can see below, it gives a greater number of eyes in the head than naturally present.

Challenges With GAN

Problem with Perspective: GANs fail to adapt to 3D objects. It doesn't understand perspective, i.e. difference between front view and back view . As we can see below, it gives flat (2D).



Challenges With GAN

Problems with Global Structures: Same as the problem with perspective, GANs do not understand a holistic structure. For example, in the bottom left image, it gives a generated image of a quadruple cow, i.e. a cow standing on its hind legs and simultaneously on all four legs. That is definitely not possible in real life!

۱. فقدان «درک فضایی» و «مفهومی»

مدل‌های GAN به‌صورت سنتی بر پایه **یادگیری توزیع پیکسل‌ها** کار می‌کنند، نه **یادگیری قواعد دنیای واقعی**.

- **مشکل پرسپکتیو:** GAN متوجه نمی‌شود که یک شیء در دوردست باید کوچک‌تر باشد یا خطوط موازی باید به یک نقطه گریز برسند. او فقط یاد می‌گیرد که ترکیب پیکسل‌های A معمولاً کنار پیکسل‌های B می‌آید. اگر در مجموعه داده‌های آموزشی، زوایای دوربین خیلی متنوع نباشند، GAN نمی‌تواند قانون پرسپکتیو را تعمیم دهد و خروجی‌هایی با پرسپکتیوهای عجیب و غریب تولید می‌کند.

۲. یادگیریِ «بافت» به جای «ساختار»

GANها در یادگیری **بافت (Texture)** بسیار عالی هستند؛ مثلاً اگر به آن بگویید «پوست»، پوست را بسیار باکیفیت و طبیعی نشان می‌دهد. اما درک **ساختار (Structure)** برایش سخت است.

- **خطاهای آناتومیک:** وقتی GAN یک انسان را می‌سازد، ممکن است تعداد انگشتان دست را اشتباه کند یا فرم صورت را در زوایای مختلف به هم بریزد. دلیلش این است که Discriminator فقط در یک نگاه کلی، "طبیعی بودن کلیِ تصویر" را قضاوت می‌کند و ممکن است متوجه نشود که مثلاً فرم چشم با زاویه سر هماهنگ نیست.

۳. مشکل "بافت‌های نامنسجم" (Artifacts)

این همان چیزی است که باعث می‌شود عکس‌های GAN قدیمی "مصنوعی" به نظر برسند.

- در تصاویر تولیدی توسط GANها، اغلب شاهد **الگوهای تکرارشونده کوچک (Checkerboard Artifacts)** هستیم. این به دلیل نحوه کارکرد لایه‌های ریاضی (Convolution) در شبکه تولیدکننده است که سعی می‌کند با بزرگ‌نمایی‌های ریاضی، تصویر را بسازد و گاهی این لایه‌ها نمی‌توانند پیوستگی کامل و نرمی را در کل تصویر حفظ کنند.

۴. نبود فرآیند اصلاح‌کننده (Refinement)

برخلاف مدل‌های Diffusion که تصویر را در صدها مرحله «اصلاح و تصحیح» می‌کنند (هر مرحله نویزها را پاک می‌کند و ساختار را دقیق‌تر می‌کند)، GANها تصویر را در **یک مرحله** می‌سازند.

- وقتی GAN تصویر را تولید می‌کند، فرصتی برای اصلاح خطاهای ساختاری ندارد. اگر در همان ابتدایِ تولیدِ پیکسل‌ها، یک اشتباه کوچک (مثلاً کج شدن یک خط) رخ دهد، تا انتهای فرآیند باقی می‌ماند و حتی ممکن است بزرگ‌تر شود.

۵. مشکل تعمیم‌پذیری (Generalization)

GANها تمایل دارند داده‌های آموزشی را حفظ (Memorize) کنند. اگر یک شیء را از زاویه‌ای که در داده‌های آموزشی نبوده بخواهید، GAN سعی می‌کند آن را از روی حدس‌های ناقص خود بسازد. چون GAN «مدل سه‌بعدی» از اشیاء ندارد (برخلاف انسان که درک فیزیکی از اشیاء دارد)، وقتی می‌خواهد پرسپکتیو را تغییر دهد، عملاً «خیال‌پردازی» می‌کند و چون پایه‌ای در قوانین فیزیک ندارد، نتیجه غیرمنطقی می‌شود.

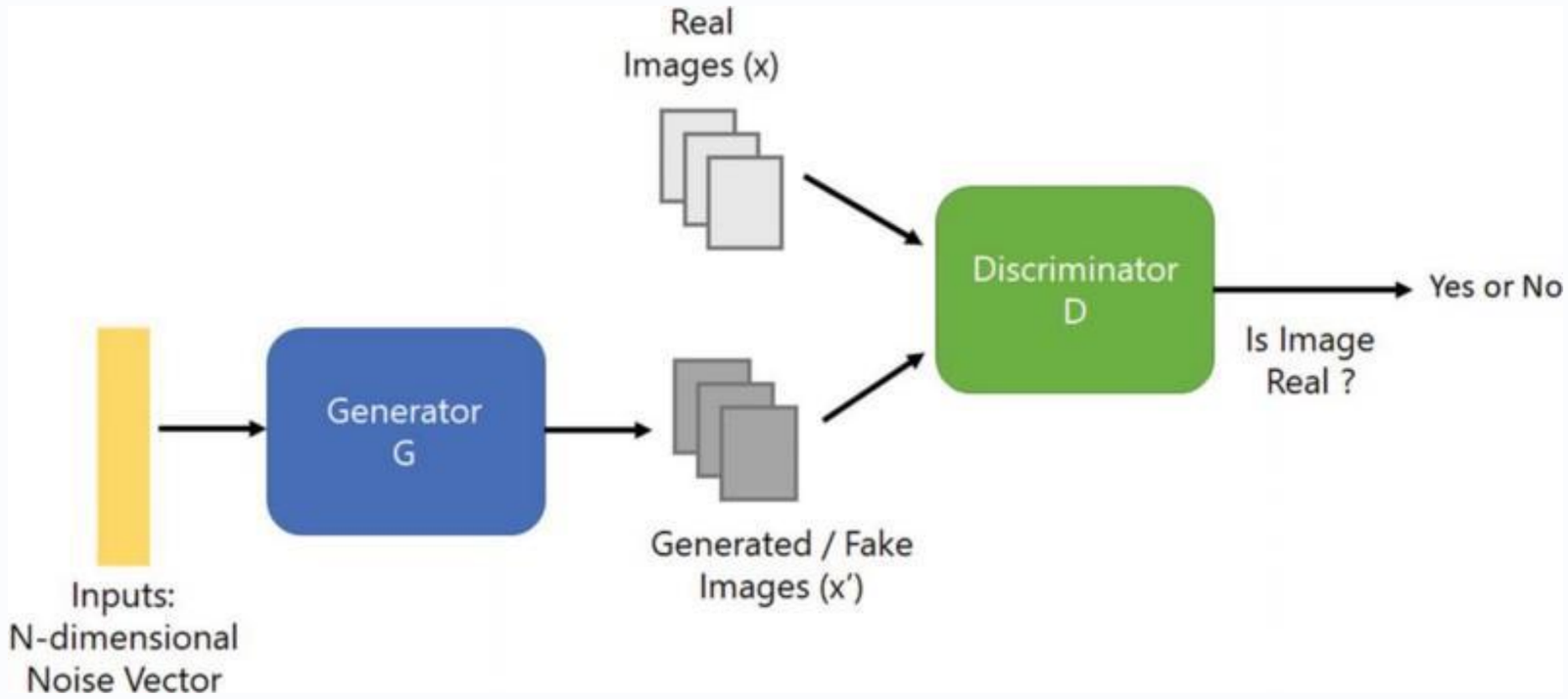
Some Extensions on GANs

1. **DCGANs**
2. **BGAN**
3. **SRGAN**
4. **CGAN**
5. **Auxiliary Classifier GAN**
6. **Semi Supervised GAN**
7. **StackGAN**
8. **Disco GANS**
9. **Flow based GANs**
10. **InfoGANs**
11. Wasserstein GAN
12. Bidirectional GAN
13. Context Conditional GAN
14. Context Encoder
15. Coupled GANs
16. CycleGAN
17. DualGAN
18. LSGAN
19. Pix 2 Pix
20. PixelDA
21. Wasserstein GAN GP
22. Adversarial Autoencoder

(1) Deep Convolutional GANs (DCGANs)

https://arxiv.org/abs/1710.10196 (Jan 2016)

In this article, we will see how a neural net maps from random noise to an image matrix and how using Convolutional Layers in the generator network produces better results.

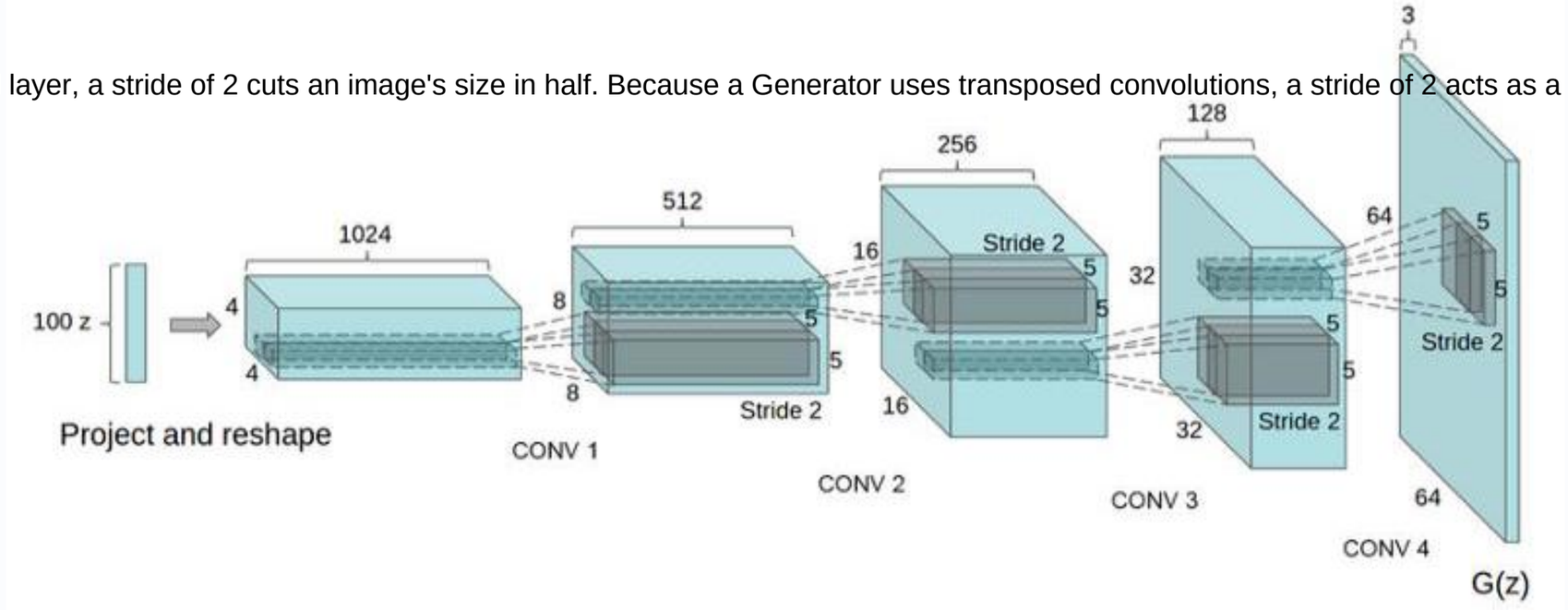


Original DCGAN architecture (Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks) have *four* convolutional layers for the **Discriminator** and *four* "four fractionally-strided convolutions" layers for the **Generator**.

(DCGAN) Generator

This network takes in a 100 x 1 noise vector, denoted z, and maps it into the G(Z) output which is 64 x 64 x 3. This architecture is especially interesting the way the first layer expands the random noise. The network goes from 100 x 1 to 1024 x 4 x 4; This layer is denoted ‘project and reshape’. We see that following this layer, classical convolutional layers are applied. In the diagram above we can see that the N parameter, (Height/Width), goes from 4 to 8 to 16 to 32 , it doesn't appear that there is any padding, the kernel filter parameter F is 5 x 5 , and the stride is 2. You may find this equation to be useful for designing your own convolutional layers for customized output sizes.

In a standard convolutional layer, a stride of 2 cuts an image's size in half. Because a Generator uses transposed convolutions, a stride of 2 acts as a multiplier, doubling the spatial



we see the network goes from:

100 x 1 → 1024 x 4 x 4 → 512 x 8 x 8 → 256 x 16 x 16 → 128 x 32 x 32 → 64 x 64 x 3

Deep Convolutional GANs (DCGANs)

Above is the output from the network presented in the paper, citing that this came after 5 epochs of training. Pretty impressive stuff.

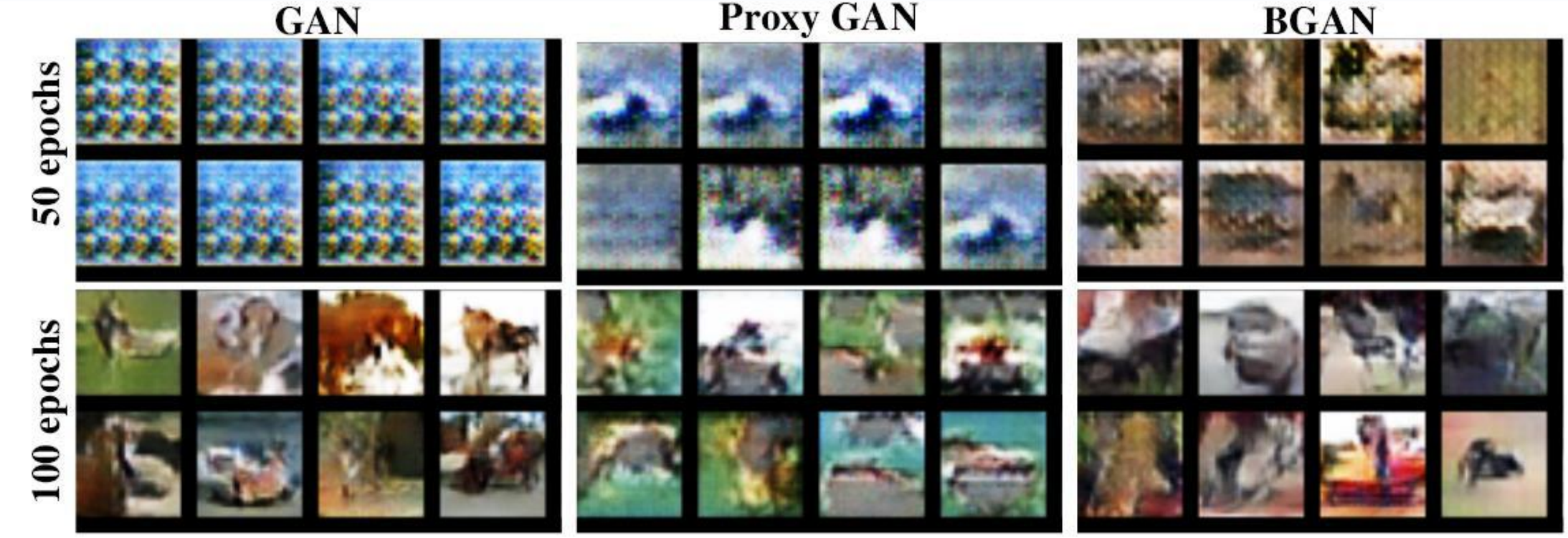
(2) Boundary-Seeking Generative Adversarial Networks (BGAN)

https://arxiv.org/abs/1702.08431 2017

- We introduce a method for training GANs with discrete data that uses the estimated difference measure from the discriminator to compute importance weights for generated samples, thus **providing a policy gradient for training the generator**.
- The importance weights have **a strong connection to the decision boundary of the discriminator**, and we call our method boundary-seeking GANs (BGANs).
- We demonstrate the effectiveness of the proposed algorithm with **discrete image and character-based natural language generation**.
- In addition, the boundary-seeking objective extends to continuous data, which can be used to **improve stability of training**, and we demonstrate this on Celeba , Large-scale Scene Understanding (LSUN) bedrooms, and ImageNet without conditioning.

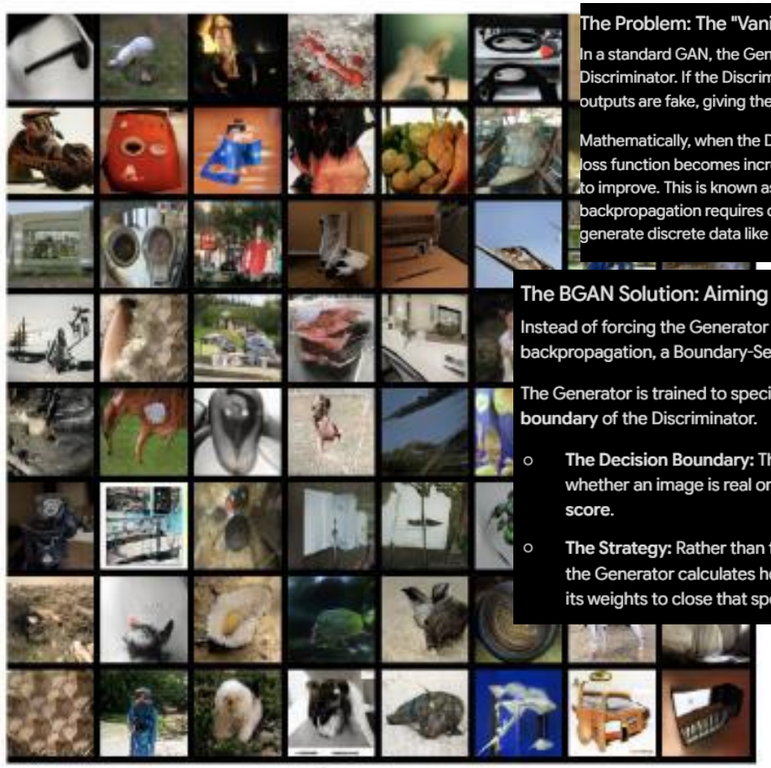
Boundary-Seeking Generative Adversarial Networks (BGAN)

Training a GAN with different generator loss functions and 5 updates for the generator for every update of the discriminator.
Over-optimizing the generator can lead to instability and poorer results depending on the generator objective function.
Samples for GAN and GAN with the proxy loss are quite poor at 50 discriminator epochs (250 generator epochs), while BGAN is noticeably better.
At 100 epochs, these models have improved, though are still considerably behind BGAN.



Boundary-Seeking Generative Adversarial Networks (BGAN)

CelebA



Imagenet

LSUN

Figure 3: Highly realistic samples from a generator trained with BGAN on the CelebA and LSUN datasets. These models were trained using a deep ResNet architecture with gradient norm regularization (Roth et al., 2017). The Imagenet model was trained on the full 1000 label dataset without conditioning.

The Problem: The "Vanishing Gradient" and Discrete Data

In a standard GAN, the Generator learns by receiving gradients passed back through the Discriminator. If the Discriminator is highly accurate, it can easily tell that the Generator's outputs are fake, giving them a score close to 0.

Mathematically, when the Discriminator's output $D(G(z))$ hits 0, the slope (gradient) of the loss function becomes incredibly flat. The Generator receives practically zero feedback on *how* to improve. This is known as the **vanishing gradient problem**. Furthermore, because standard backpropagation requires continuous, differentiable outputs, standard GANs struggle to generate discrete data like text tokens.

The BGAN Solution: Aiming for the Decision Boundary

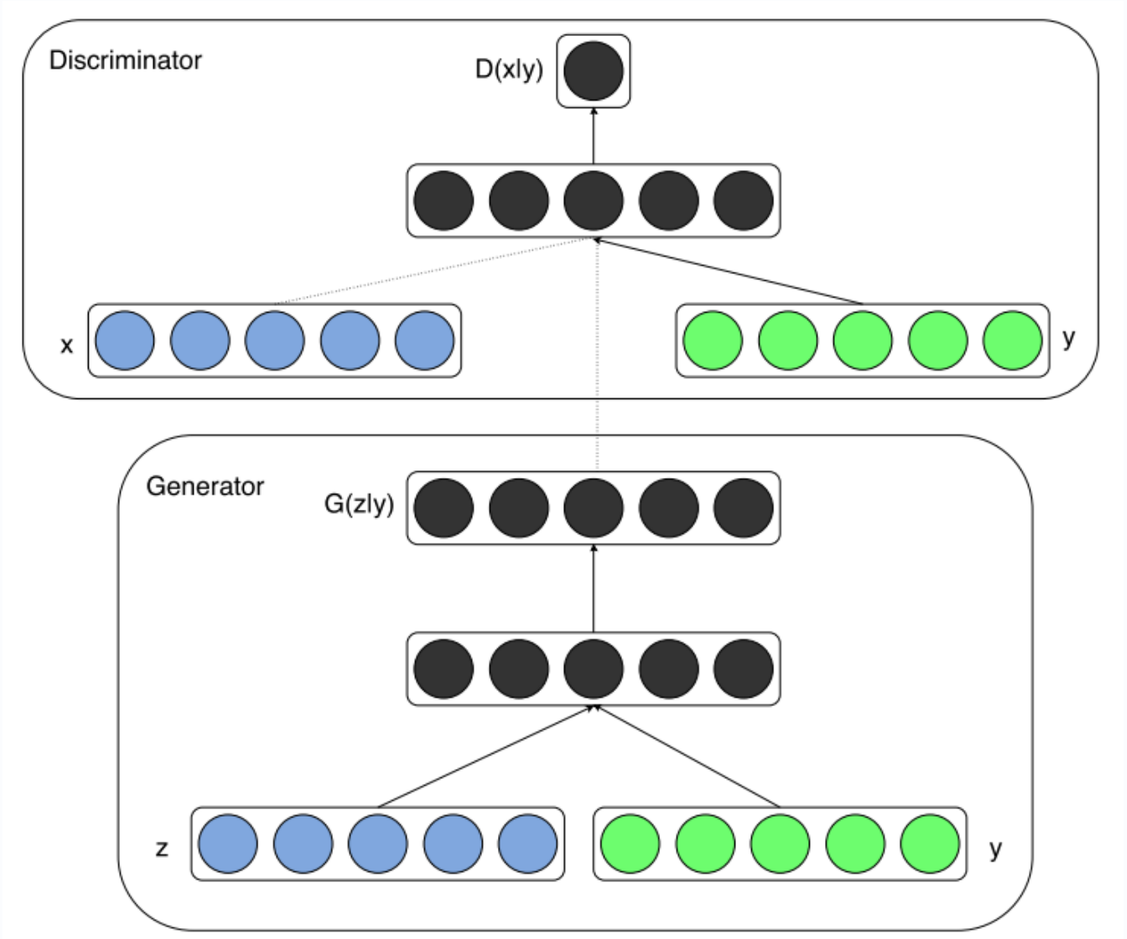
Instead of forcing the Generator to blindly maximize the Discriminator's error via traditional backpropagation, a Boundary-Seeking GAN changes the objective.

The Generator is trained to specifically generate samples that sit directly on the **decision boundary** of the Discriminator.

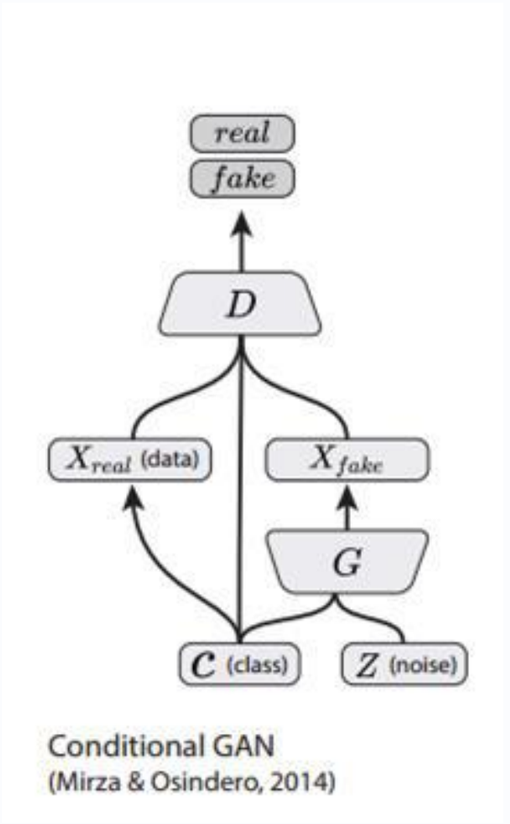
- **The Decision Boundary:** This is the line where the Discriminator is completely unsure whether an image is real or fake—the point where it outputs exactly a **0.5 probability score**.
- **The Strategy:** Rather than trying to completely fool the Discriminator into outputting a 1, the Generator calculates how far its current output is from this 0.5 boundary and shifts its weights to close that specific gap.

(3) Conditional GANs (cGANs)

Mehdi Mirza, Simon Osindero (Submitted on 6 Nov 2014), Source: https://arxiv.org/pdf/1411.1784.pdf



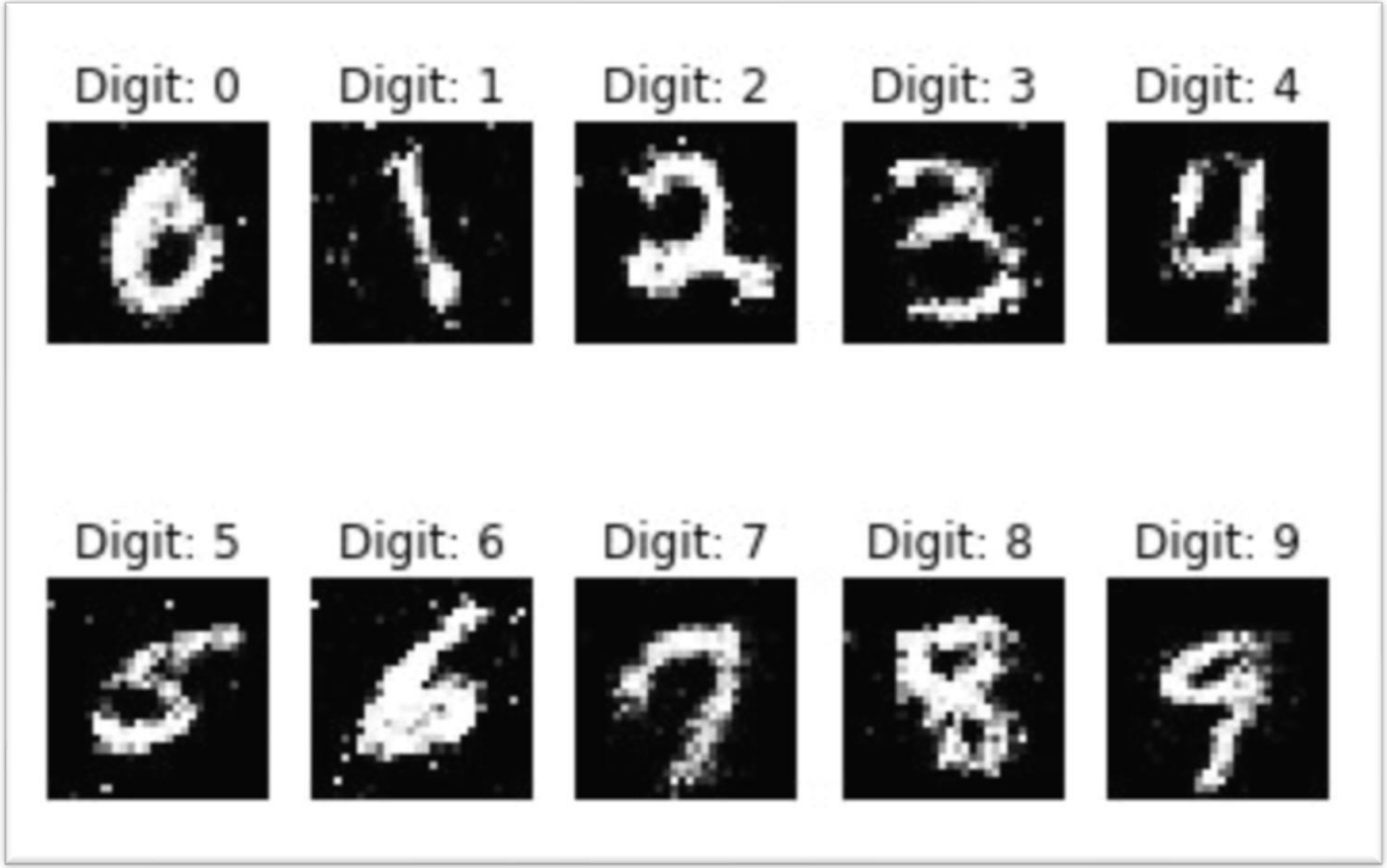
1. These GANs use extra label information and result in better quality images and are able to control how generated images will look. cGANs learn to produce better images by exploiting the information fed to the model.
2. It does give the end user a mechanism for controlling the Generator output



Conditional GAN
(Mirza & Osindero, 2014)

Conditional GANs (cGANs)

The results of Conditional GANs are very impressive. They allow for much greater control over the final output from the generator.

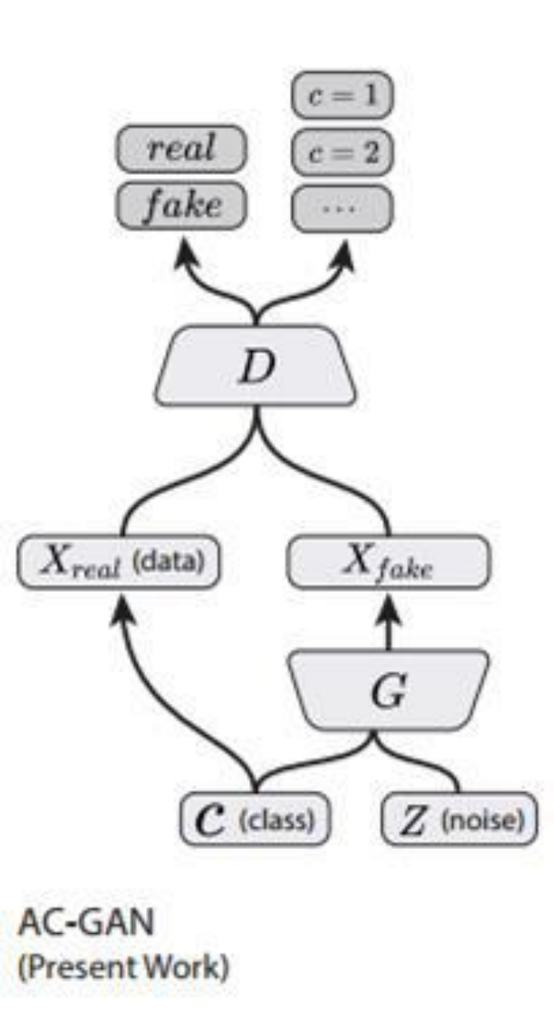


Results testing Conditional GANs on MNIST

AC-GAN (Auxiliary Classifier GANs)

By adding an auxiliary classifier to the discriminator of a GAN, the discriminator produces not only a probability distribution over sources but also probability distribution over the class labels.

Source: Augustus Odena , Christopher Olah , Jonathon Shlens . Conditional Image Synthesis with Auxiliary Classifier GANs. 2016



The sample generated images from CIFAR 10 dataset.

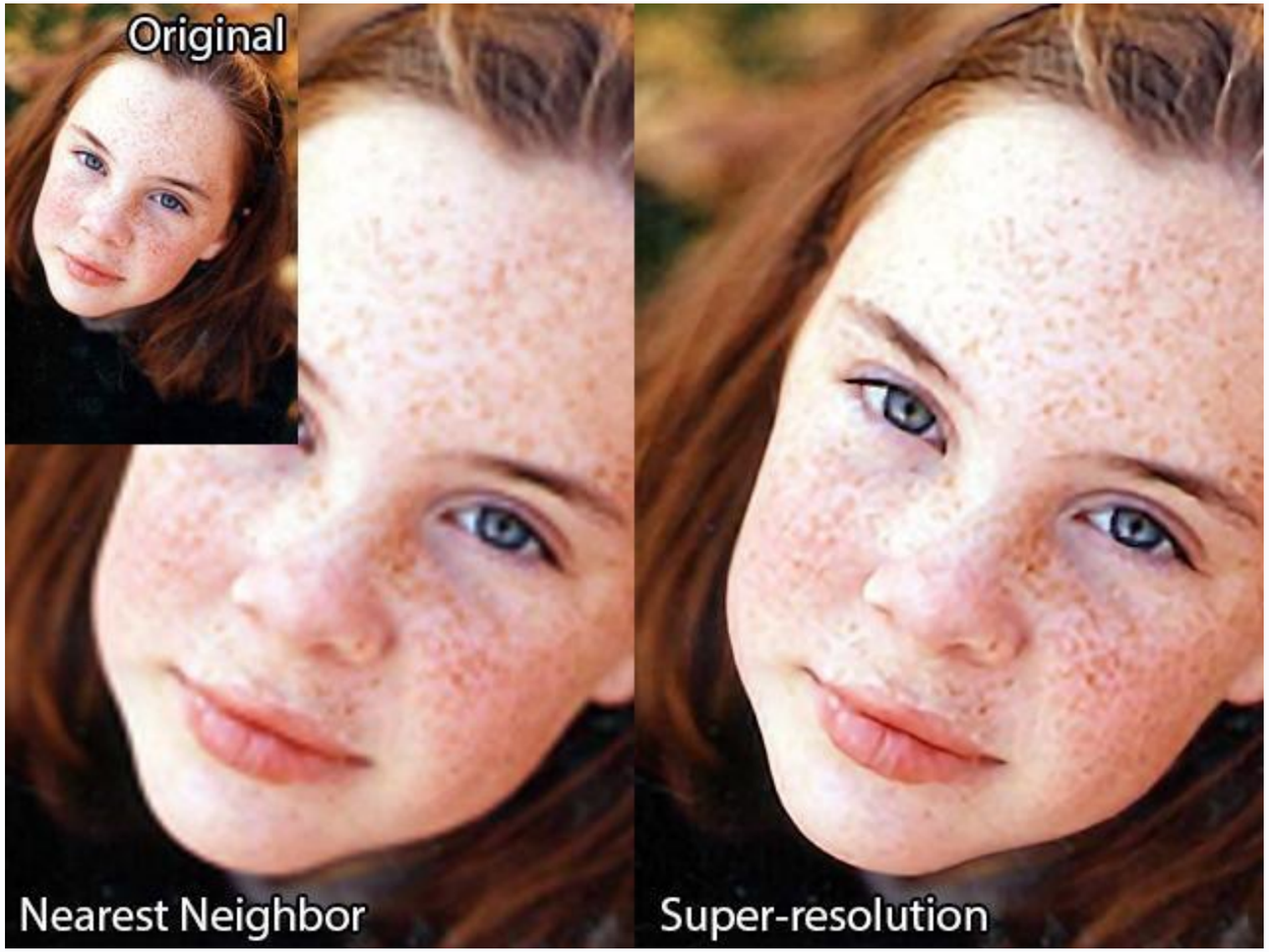
AC-GAN (Auxiliary Classifier GANs)

In the AC-GAN paper, 100 different GAN models each handle 10 different classes from the ImageNet dataset consisting of 1,000 different object categories.

The sample generated images from ImageNet dataset.

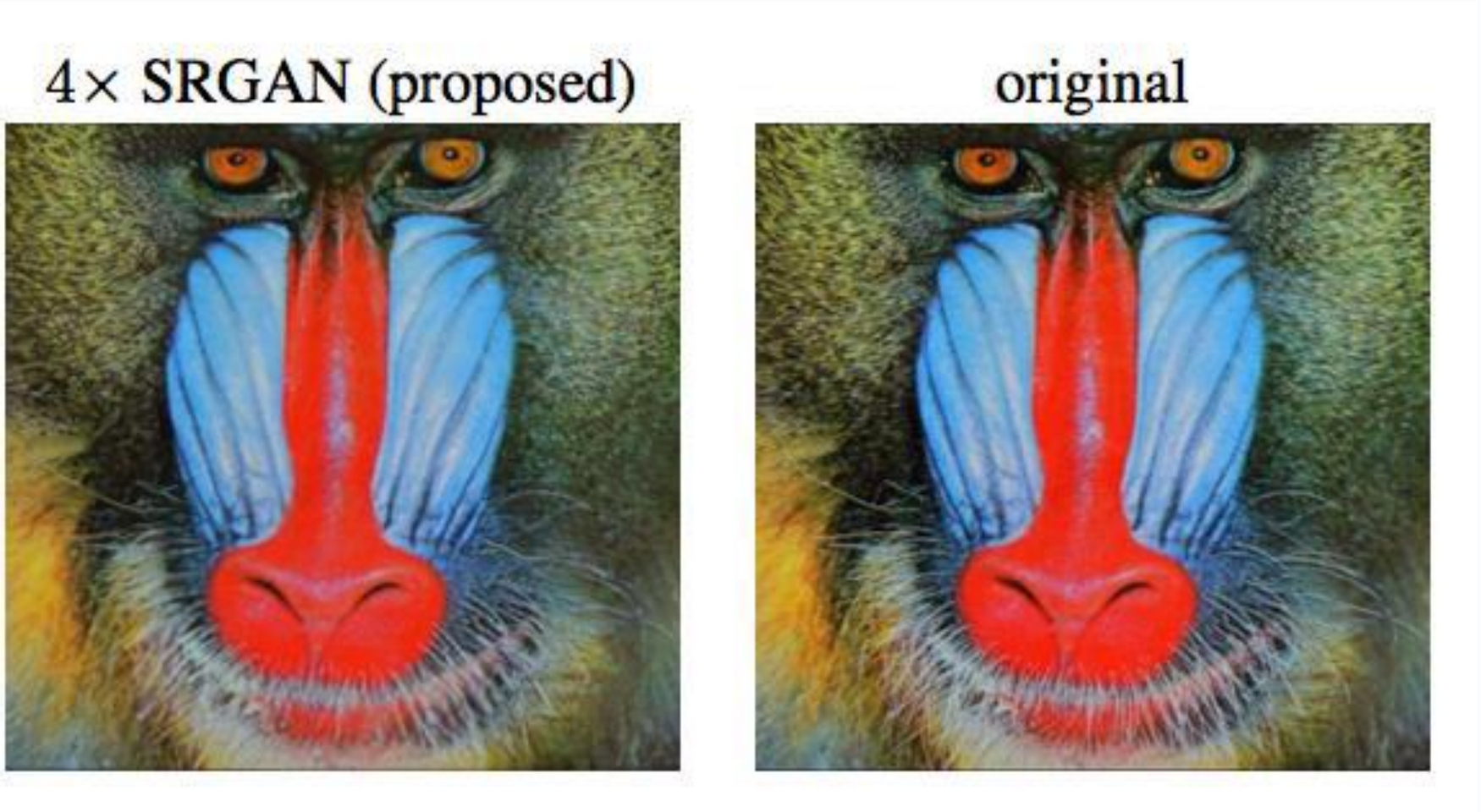
(4) Super Resolution GAN (SR GAN)

Super resolution is a task concerned with upscaling images from low resolution sizes such as 90 x 90, into high resolution sizes such as 360 x 360



Source: “**Photo Realistic Single Image Super Resolution Using a Generative Adversarial Network.**”, Christian Ledig, Lucas Theis, Ferenc Huszar, Jose Caballero, Andrew Cunningham, Alejandro Acosta, Andrew Aitken, Alykhan Tejani, Johannes Totz, Zehan Wang, Wenzhe Shi.

Super Resolution GAN (SR GAN)



In this example, 90 x 90 to 360 x 360 is denoted as an up-scaling factor of 4x.

These networks learn a mapping from the low-resolution patch through a series of convolutional, fully-connected, or transposed convolutional layers into the high-resolution patch.

Super Resolution GAN (SR GAN)

For example, this network could take a 64 x 64 low-resolution patch, convolve over it a couple times such that the feature map is something like 16 x 16 x 512, flatten it into a vector, apply a couple of fully-connected layers, reshape it, and finally, up-sample it into a 256 x 256 high-resolution patch through transposed convolutional layers.

(5) StackGAN

Text to Photo-realistic Image Synthesis with Stacked generative Adversarial Networks, Han Zhang, ...

The authors of this paper propose **a solution to the problem of synthesizing high quality images from text descriptions in computer vision**. They propose Stacked Generative Adversarial Networks (StackGAN) to generate 256 x 256 photo-realistic images conditioned on text descriptions. They decompose the hard problem into more manageable sub-problems through a sketch refinement process.

stage 1 : it creates a low resolution image base on the text described,
stage 2: take output of stage 1 and text as input and generate high resolution image.

The Stage-I GAN sketches the primitive shape and colors of the object based on the given text description, yielding Stage-I low-resolution images. The Stage-II GAN takes Stage-I results and text descriptions as inputs, and generates high-resolution images with photo-realistic details.

The architecture of the proposed StackGAN

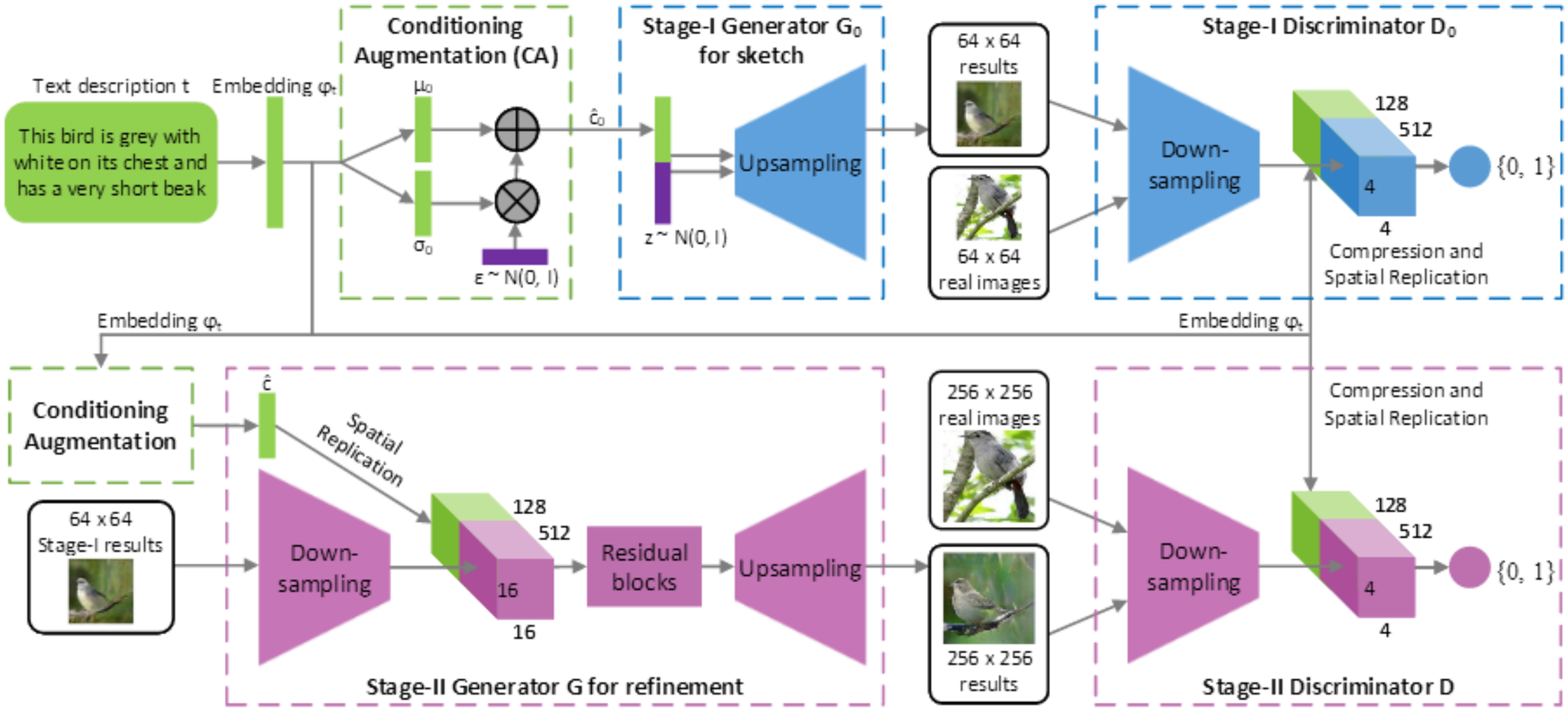


Figure 2. The architecture of the proposed StackGAN. The Stage-I generator draws a low-resolution image by sketching rough shape and basic colors of the object from the given text and painting the background from a random noise vector. Conditioned on Stage-I results, the Stage-II generator corrects defects and adds compelling details into Stage-I results, yielding a more realistic high-resolution image.

StackGAN

Comparison

Figure 3. Example results by our StackGAN, GAWWN [24], and GAN-INT-CLS [26] conditioned on text descriptions from CUB test set.

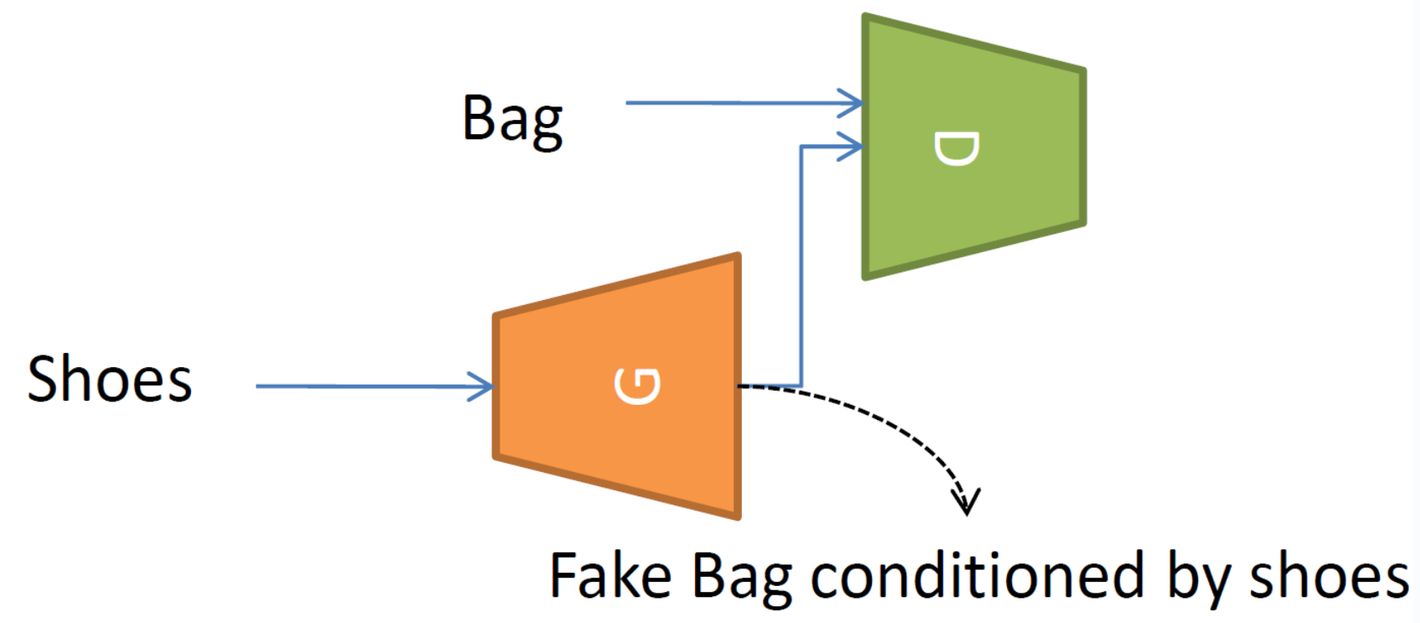
StackGAN

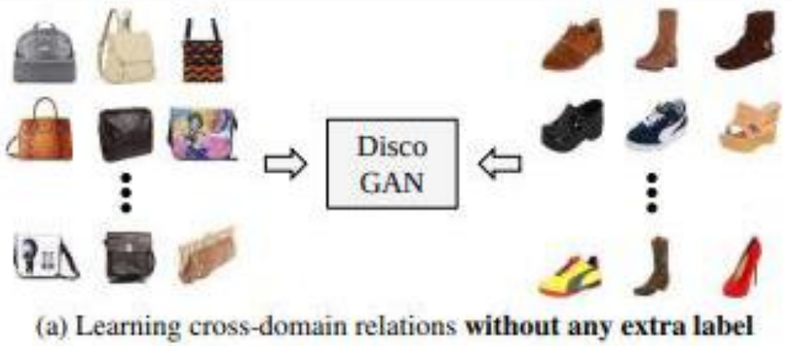
Comparison

Figure 4. Example results by our StackGAN and GAN-INT-CLS [26] conditioned on text descriptions from Oxford-102 test set (leftmost four columns) and COCO validation set (rightmost four columns).

(6) Discover Cross Domain Relations with GANs (Disco GANS)

The authors of this paper propose a method based on generative adversarial networks that learns **to discover relations between different domains (without any extra labels)**. Using the discovered relations, the network transfers style from one domain to another.





(a) Learning cross-domain relations **without any extra label**

(b) Handbag images (input) & **Generated** shoe images (output)

(c) Shoe images (input) & **Generated** handbag images (output)

(7) Flow Based GANs

Oct 13, 2018 by Lilian Weng

Here is a quick summary of the difference between GAN, VAE, and flow-based generative models:

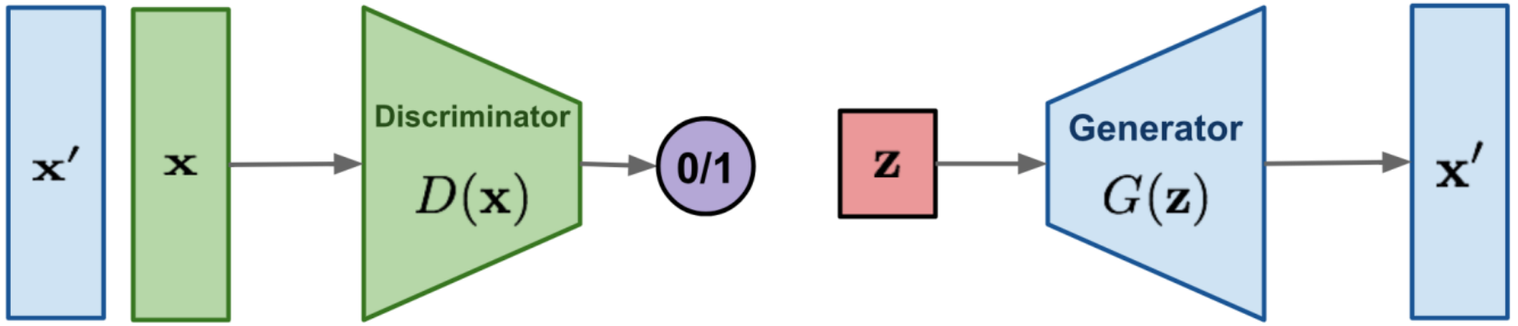
1. Generative adversarial networks: GAN provides a smart solution to model the data generation, an unsupervised learning problem, as a supervised one. The discriminator model learns to distinguish the real data from the fake samples that are produced by the generator model. Two models are trained as they are playing a minimax game.

2. Variational autoencoders: VAE inexplicitly optimizes the log-likelihood of the data by maximizing the evidence lower bound (ELBO).

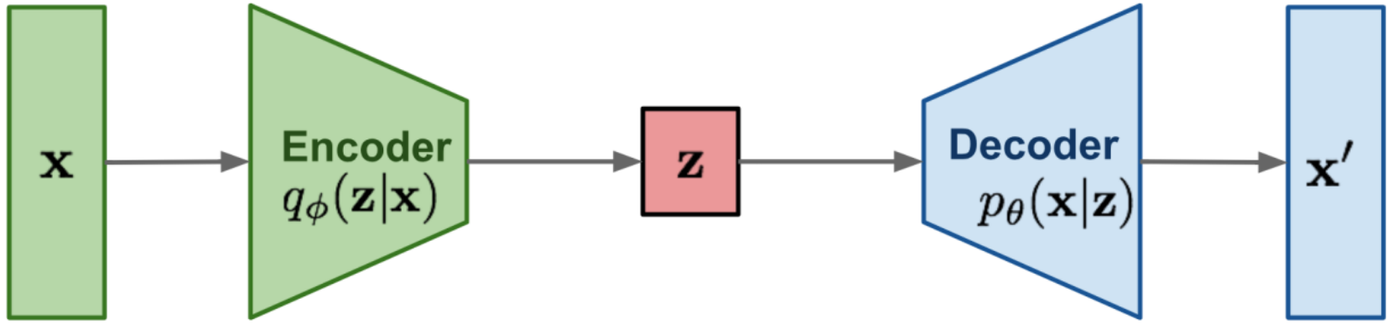
3. Flow-based generative models: A flow-based generative model is constructed by a sequence of invertible transformations. Unlike other two, the model explicitly learns the data distribution $p(x)$ and therefore the loss function is simply the negative log-likelihood.

Types of Generative Models

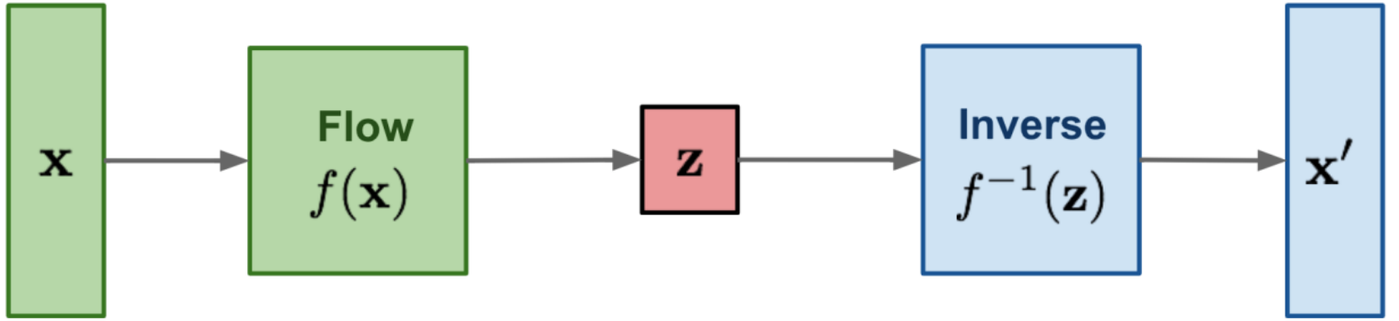
GAN: minimax the classification error loss.



VAE: maximize ELBO.



Flow-based generative models: minimize the negative log-likelihood



Flow Based GANs

Toward using both **maximum likelihood** and **adversarial** training

1. Implicit models such as generative adversarial networks (GAN) **often generate better samples** compared to **explicit models trained by maximum likelihood**.

2. However, we know that the method based on **maximum likelihood** explicitly learn the probability density function of the input data.

3. To bridge this gap, we propose Flow GANs, a generative adversarial network for which we can perform Exact likelihood evaluation, thus **supporting both adversarial and maximum likelihood** training.

1. Core Architecture and the "Flow" ($f(\mathbf{x})$)

Unlike a GAN generator that maps random noise into an image without knowing the exact probability density, a Flow-based model maps a real data point $\mathbf{x}$ to a latent space representation $\mathbf{z}$ using a sequence of invertible functions.

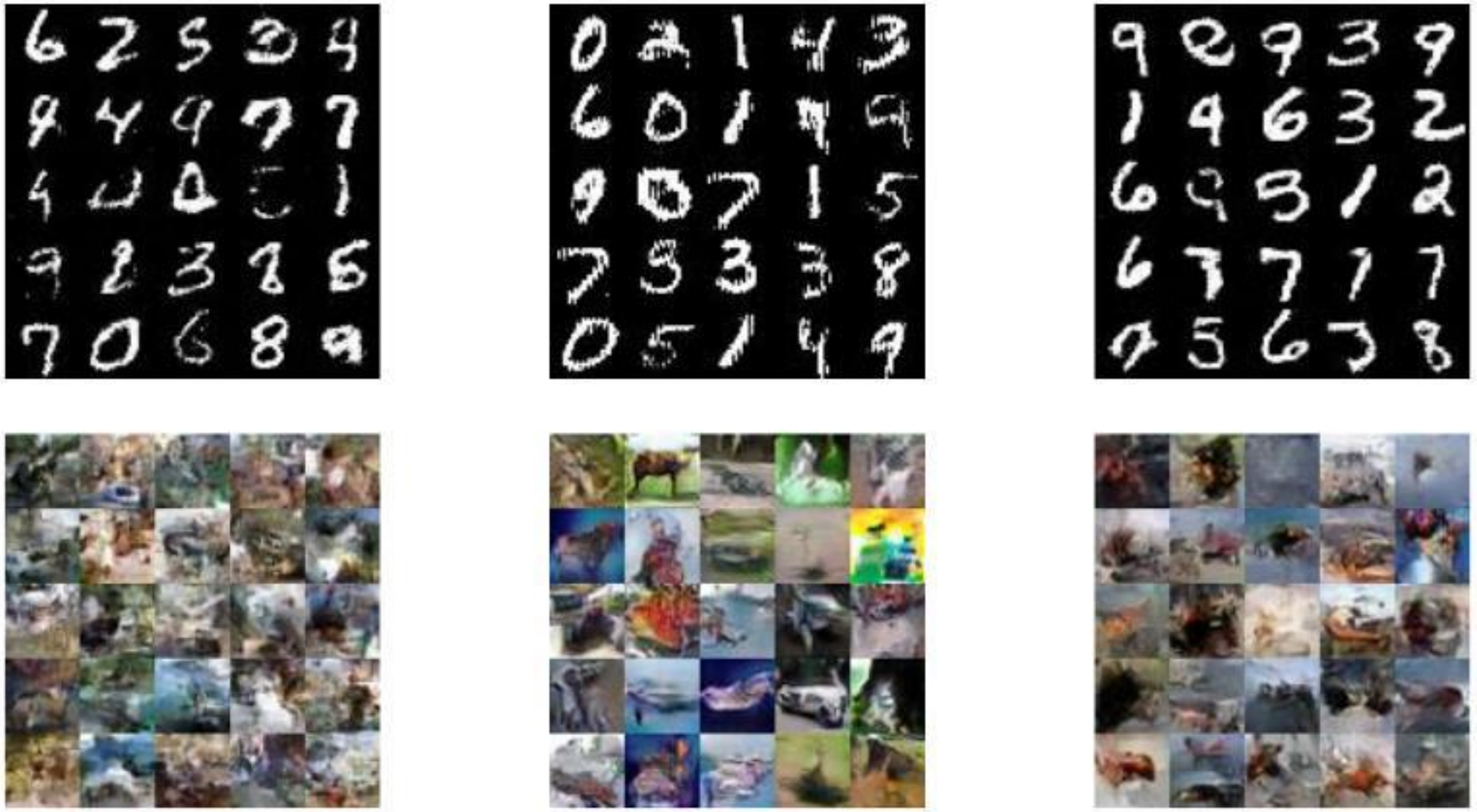
- **The Input ($\mathbf{x}$):** High-dimensional real data (like a real image).
- **The Normalizing Flow ($f(\mathbf{x})$):** The green block represents a series of invertible, differentiable operations. It pushes the complex, messy distribution of real data through these steps to "normalize" it into a simple, structured target distribution—usually a standard Gaussian distribution ($z \sim \mathcal{N}(0, I)$).
- **The Latent Variable ($\mathbf{z}$):** The resulting exact noise vector in latent space.

2. Exact Generation via the Inverse Flow ($f^{-1}(\mathbf{z})$)

Because every single layer inside the "Flow" block is mathematically designed to be 100% reversible, the exact same network can be run completely backward.

- **Sampling:** To generate a completely new data point, you randomly sample a vector $\mathbf{z}$ from the simple Gaussian noise distribution.
- **The Inverse Block ($f^{-1}(\mathbf{z})$):** The blue block takes that noise and pushes it backward through the exact mathematical inverse of the training layers.
- **The Output ($\mathbf{x}'$):** This directly yields a reconstructed or newly generated sample $\mathbf{x}'$ in the original data space.

Flow Based GANs



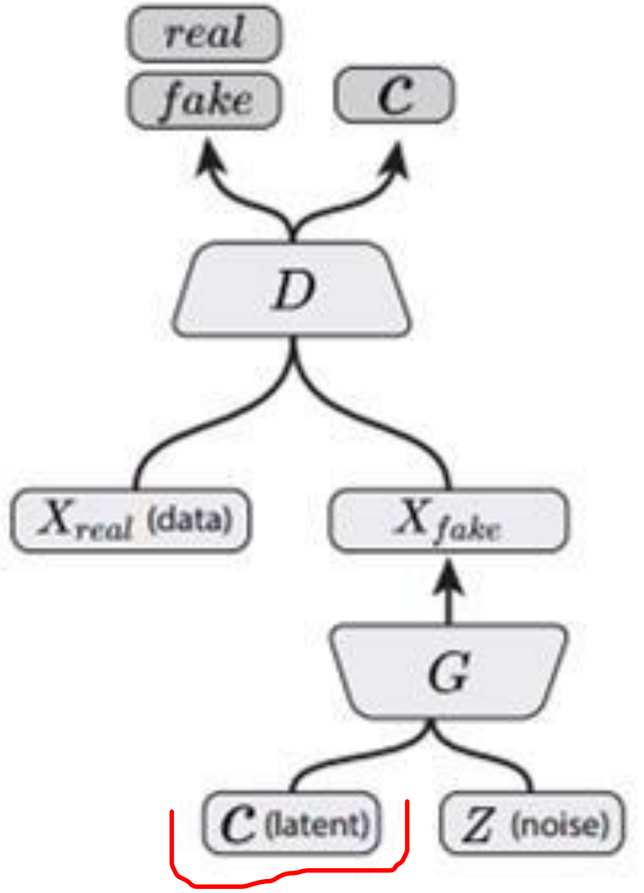
(a) MLE (b) ADV (c) Hybrid

Figure 1: Samples generated by Flow-GAN models with different objectives for MNIST (**top**) and CIFAR-10 (**bottom**).

(8) InfoGANs

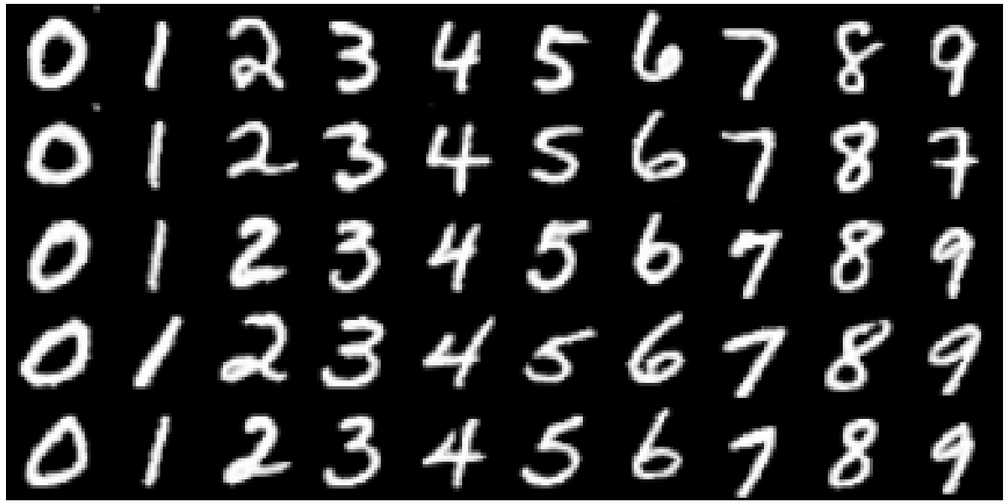
Interpretable Representation Learning by Information Maximizing Generative Adversarial Nets, NIPS 2016

InfoGAN is **an information theoretic extension to the GAN** that is able to learn **disentangled representations** in an unsupervised manner. InfoGANs are used when your dataset is very complex, when you'd like to train a cGAN and the dataset is not labelled , and when you'd like to see the most important features of your images.

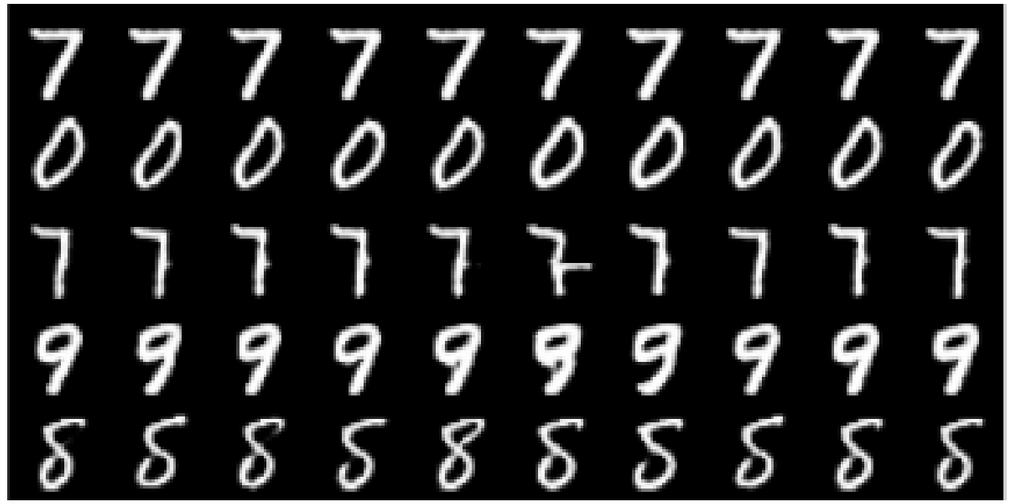


InfoGAN
(Chen, et al., 2016)

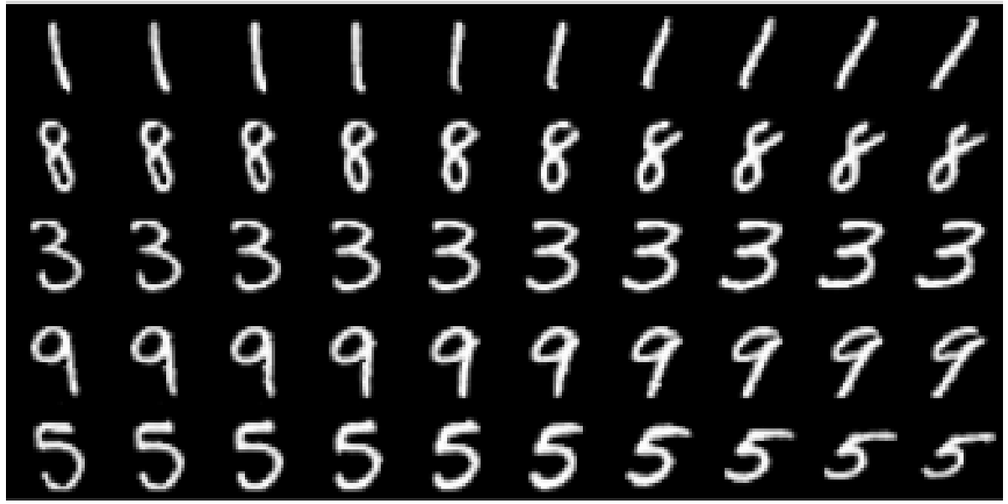
InfoGANs



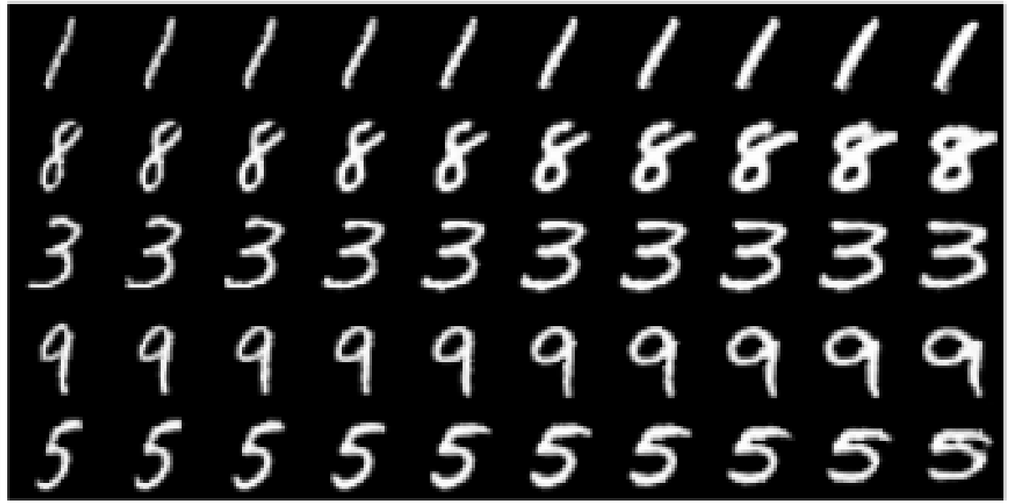
(a) Varying c_1 on InfoGAN (Digit type)



(b) Varying c_1 on regular GAN (No clear meaning)



(c) Varying c_2 from -2 to 2 on InfoGAN (Rotation)



(d) Varying c_3 from -2 to 2 on InfoGAN (Width)

InfoGANs

(a) Azimuth (pose)

(b) Elevation

(c) Lighting

(d) Wide or Narrow

InfoGANs

(a) Rotation

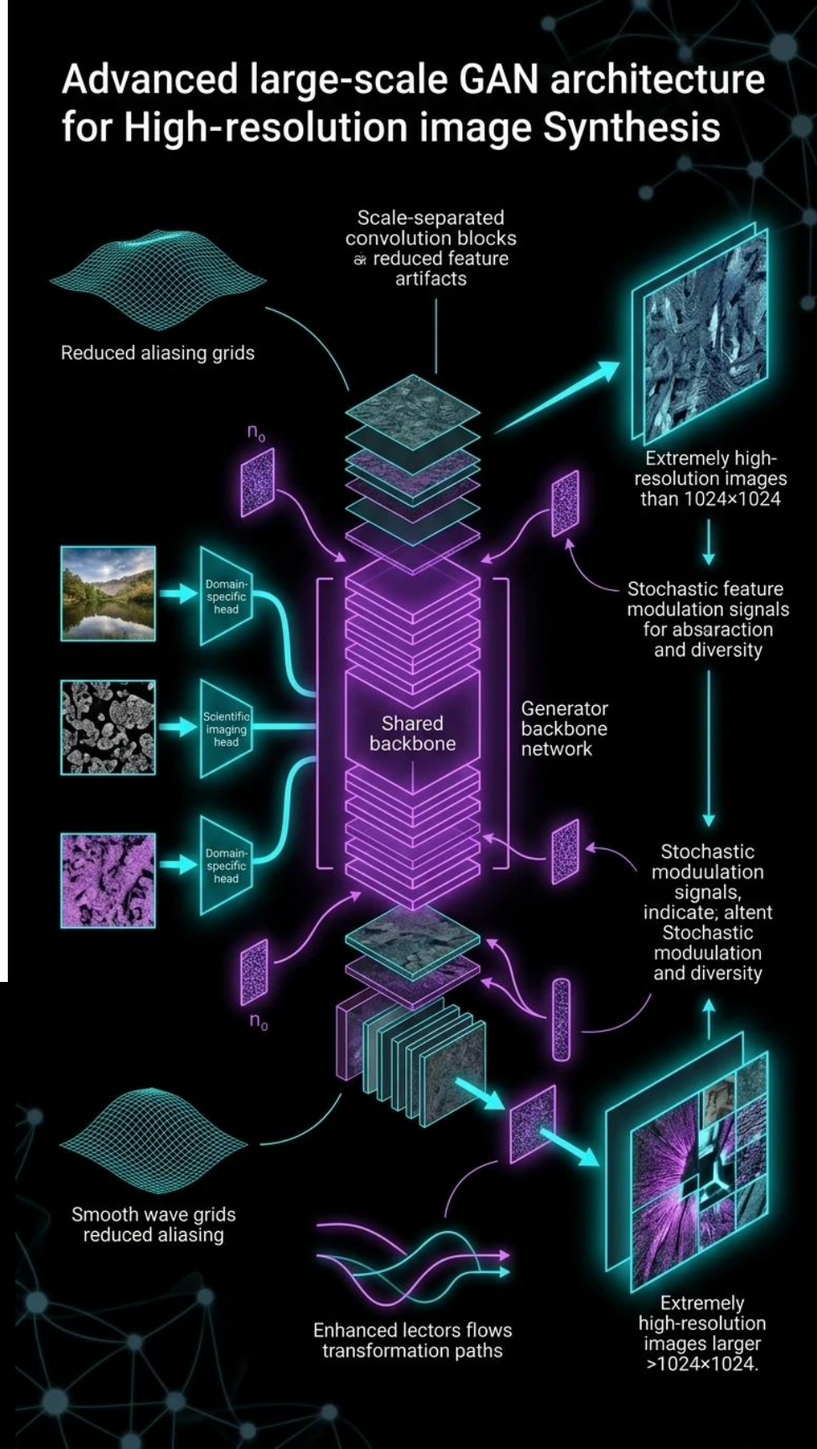
(b) Width

StyleGAN-XL (2022–2023)

Key Idea:
An advanced large-scale GAN architecture designed for high-resolution image synthesis with improved generalization across diverse visual domains.

Scientific Explanation:

- Extends StyleGAN3 with *scale-separated* convolutional blocks that reduce aliasing artifacts.
- Introduces *multi-domain training* using a shared generator backbone and domain-specific heads.
- Uses *stochastic feature modulation* to maintain diversity while preserving structure.
- Achieves state-of-the-art performance in natural images, scientific imaging, and data-scarce domains.
- Particularly effective at high resolutions (>1024×1024) due to enhanced latent manipulations.



C. Stochastic Feature Modulation (The Red Underline)

This is a critical stabilization technique. "Stochastic" means involving random probability.

- During training, the architecture injects random, multi-scale feature modulation signals throughout the synthesis network.
- This technique dynamically alters intermediate style weights during the generation process. It forces the model to decouple an object's **global geometry** from its **local textures**. This keeps the overall structural framework of the image stable while ensuring high variability in micro-details, preventing the model from producing repetitive, robotic outputs.

- **Ultra-High-Resolution Synthesis ($> 1024 \times 1024$):** It handles the intensive computational requirements of generating ultra-crisp, commercial-grade imagery without tearing, blurriness, or catastrophic forgetting during training.
- **Cross-Domain Generalization:** Because of its multi-domain heads, a single deployment of StyleGAN-XL can fluidly switch between generating natural environments, portraits, and abstract graphics.
- **Scientific & Medical Imaging:** It is highly effective at synthesizing scientific data (microscopy, medical scans, cell structures) where preserving absolute spatial structure and local geometric accuracy is crucial.
- **Data-Scarce Domain Adaptation:** If you only have a few hundred real samples of a specific image type, you can freeze StyleGAN-XL's pre-trained shared backbone and quickly fine-tune only the domain-specific heads. This lets you synthesize highly realistic niche data without causing the model to overfit.

GigaGAN (Meta AI, 2023)

Key Idea:
A massively-scaled GAN (10B parameters) enabling text-to-image generation competitive with diffusion models.

Scientific Explanation:

- Trains on large-scale multimodal datasets using text as conditioning.
- Uses a *spectrally normalized* transformer-based generator to stabilize training at scale.
- Incorporates *Cross-Layer Attention Modulation (CLAM)* for stronger text–image alignment.
- Achieves superior sampling speed compared to diffusion models (10–20× faster).
- Demonstrates coherent object compositionality, long-range structure, and fine-grained textures.

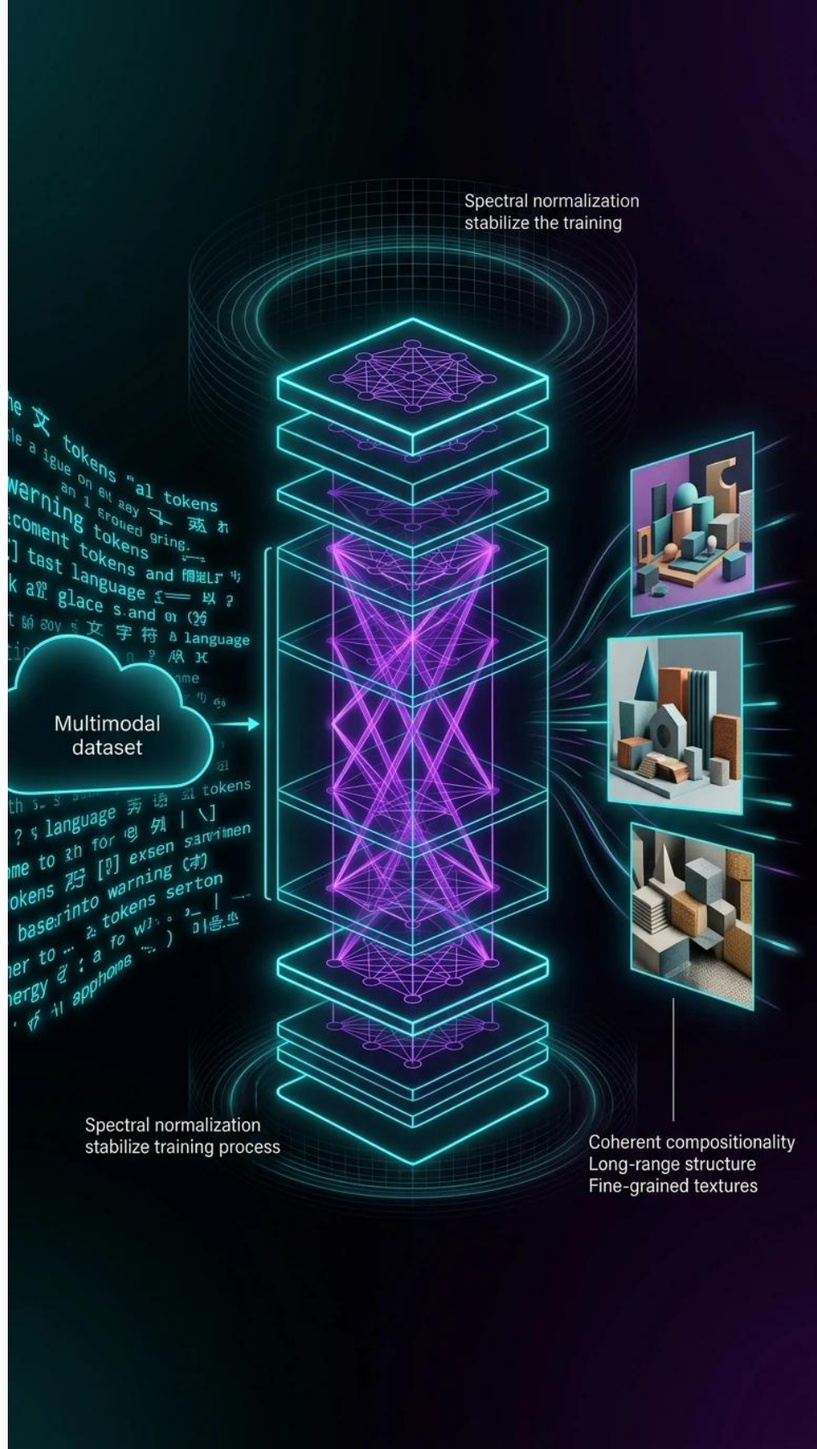
- **Transformer-Based Generator:** Instead of using classical convolutional networks as the backbone, GigaGAN swaps them for a Vision Transformer (ViT) style layout. This allows the model to process long-range spatial dependencies (how a hand on the left side of an image structurally relates to a torso on the right side).
- **Spectral Normalization:** Noted at the top and bottom of the dark diagram, this mathematical constraint bounds the weights of the network layers. It prevents the gradients from exploding, ensuring the adversarial competition between Generator and Discriminator stays stable at a 10-billion parameter scale.
- **Extreme Inference Speed (10–20× Faster):** Diffusion models must run iteratively backward through dozens of denoising steps to generate a single image, making them slow. Because GigaGAN is a GAN, it requires only a single forward pass through the generator network to produce an entire image instantly.

How CLAM Works Inside the Network:

1. **Extracting Text Tokens:** The written prompt is converted into a structured set of text token embeddings.
2. **Cross-Attention Matching:** As image features travel through the generator, CLAM applies a **cross-attention mechanism** at multiple independent layers of the network simultaneously. The image features act as the *Queries* (Q), while the text tokens act as the *Keys* (K) and *Values* (V).
3. **Multi-Scale Modulation:** Instead of forcing the text to adjust the image all at once, CLAM adaptively modulates the feature maps at different resolutions:
   - **Early Layers:** Text tokens modulate global features (e.g., *"a large red barn"* sets up the layout and primary background colors).
   - **Deep Layers:** Text tokens modulate fine-grained details (e.g., *"weathered wood texture"* dictates the microscopic patterns of the barn walls).

Why CLAM is Essential:

By providing direct, cross-layer pathways for text data, CLAM ensures **coherent object compositionality**. If you ask for *"a blue cube on top of a yellow sphere,"* CLAM keeps those concepts isolated and correctly positioned, preventing the blue and yellow attributes from bleeding into each other.

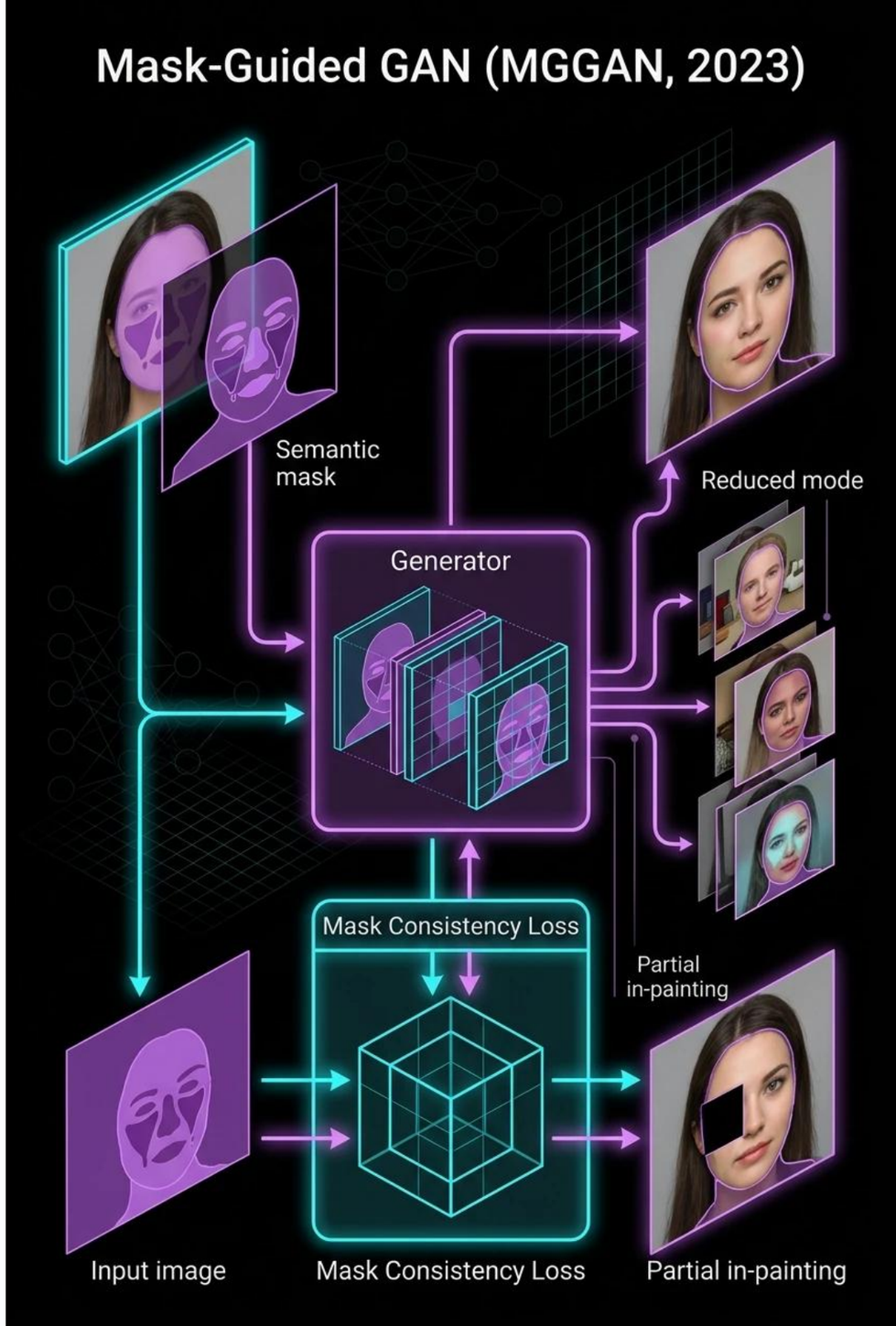


Mask-Guided GAN (MGGAN, 2023)

Key Idea:
A GAN architecture using mask supervision to produce controllable image editing and structure-aware synthesis.

Scientific Explanation:

- Generator receives both the image and its semantic mask as inputs.
- A dedicated *Mask Consistency Loss* ensures structural integrity during synthesis.
- Improves controllability in tasks like facial attribute editing, scene rearrangement, and medical image manipulation.
- Reduces mode collapse by enforcing attention on spatial regions rather than global features.
- Enables partial in-painting with structural guarantees.

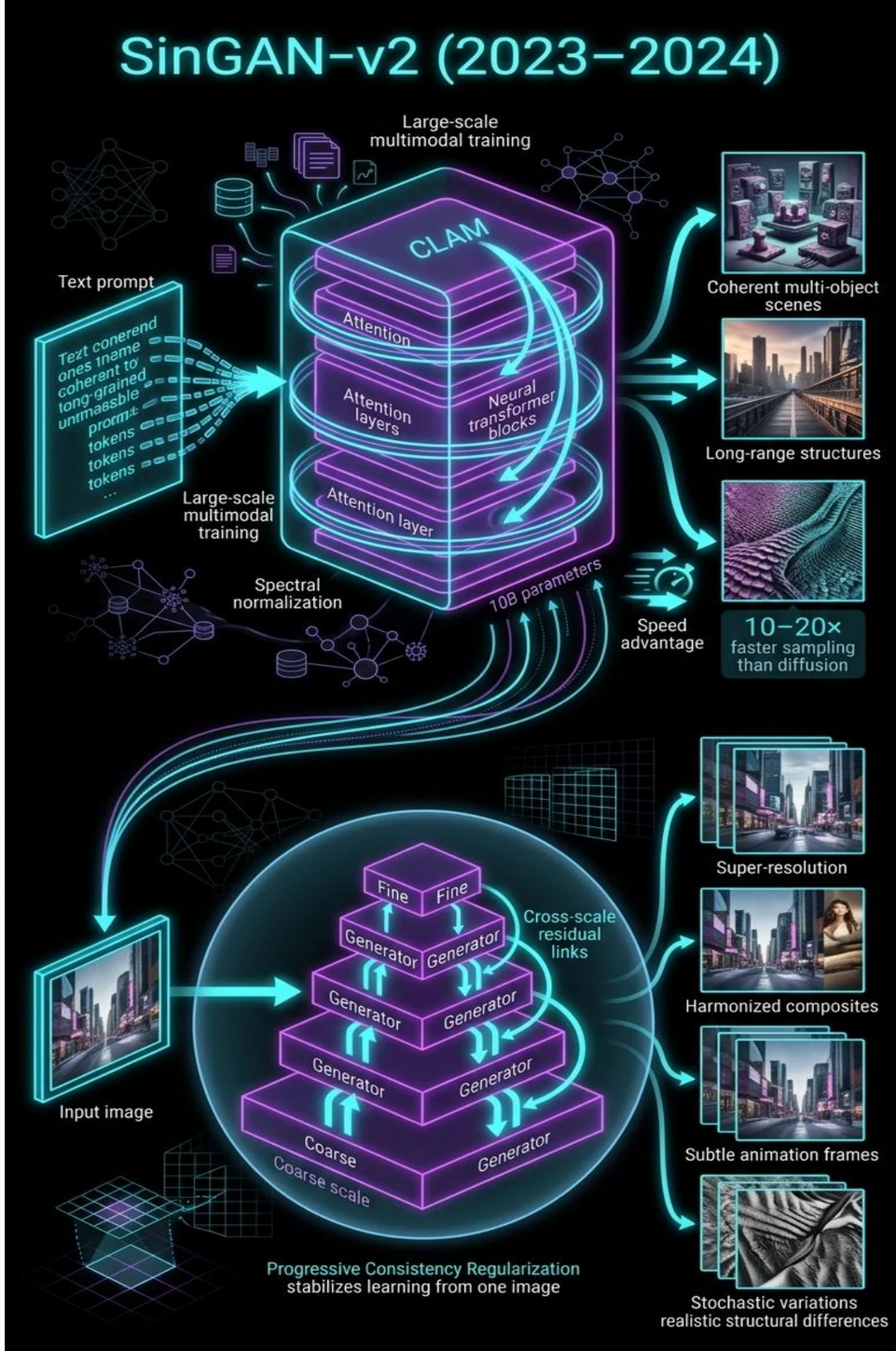


SinGAN-v2 (2023–2024)

Key Idea:
A single-image GAN that improves the original SinGAN with multi-scale feature refinement, enabling better generalization from one sample.

Scientific Explanation:

- Learns a pyramid of generators, one per scale, with cross-scale residual links.
- Introduces *Progressive Consistency Regularization* to reduce overfitting to the single training image.
- Supports image editing tasks: super-resolution, harmonization, animation, and stochastic variations.
- Achieves sharper textures and more realistic structural variations than the original SinGAN.
- Useful for extremely low-data synthesis scenarios.

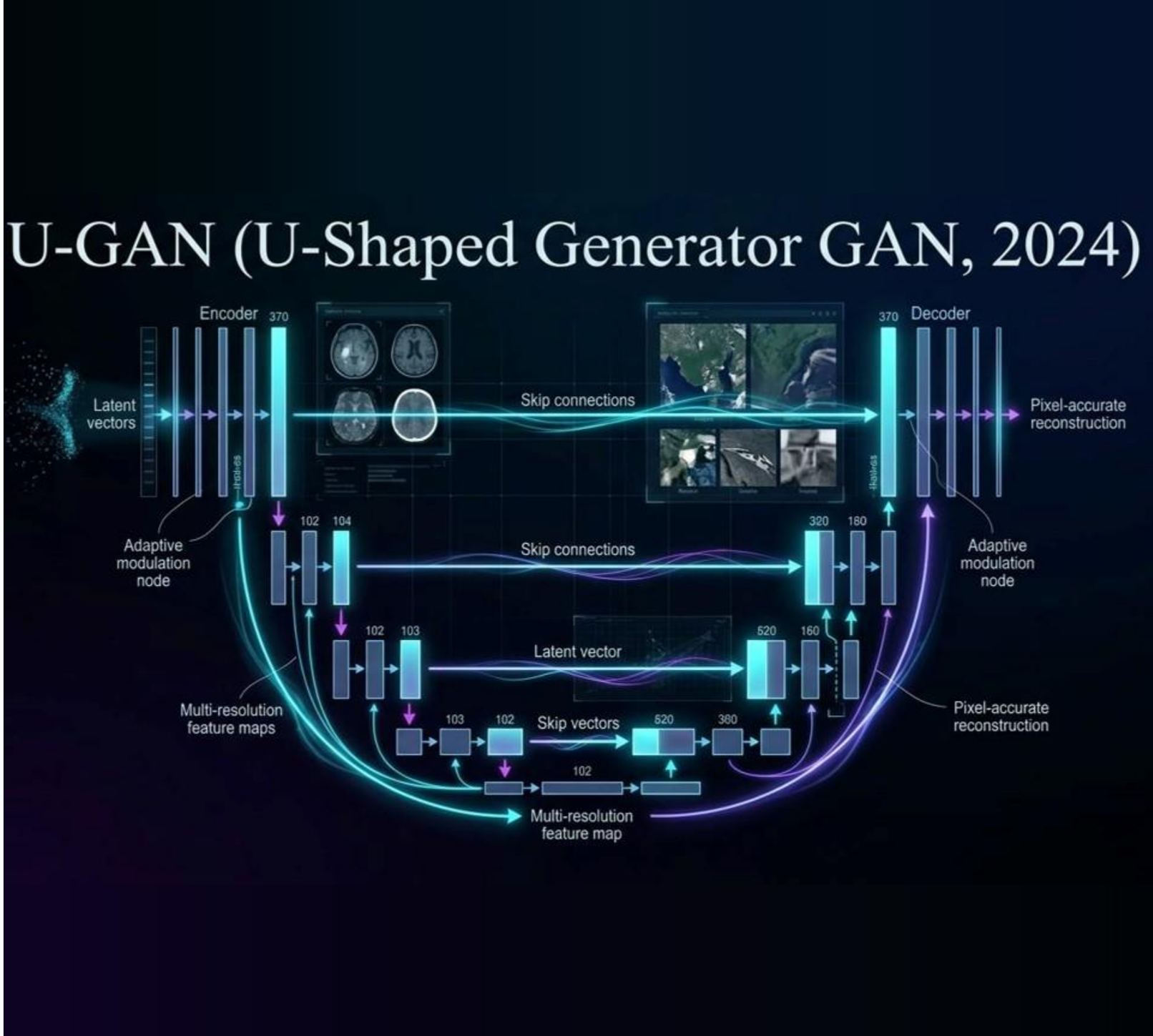


U-GAN (U-Shaped Generator GAN, 2024)

Key Idea:
A GAN architecture using a U-Net-style generator for enhanced spatial precision and multi-resolution control.

Scientific Explanation:

- Combines convolutional encoders and decoders with skip connections to preserve spatial detail.
- Provides stronger gradient flow, improving training stability with high-resolution data.
- Excels in tasks requiring pixel-accurate outputs: medical imaging, satellite imagery, restoration.
- Introduces *Adaptive Skip Modulation*, allowing latent vectors to control each resolution level independently.
- Produces higher fidelity structural consistency than traditional GAN generators.



Conclusion and Comparison

GAN Variant	Architecture (G / D)	Loss / Objective	Key Advantage	Primary Application	Key Details & Scale
GAN (2014)	MLP (flexible)	Minimax BCE: $\min_G \max_D V(D, G)$	Pioneering adversarial framework; generates highly realistic data	General fake data generation	Alternating k steps for D, then G; training can be unstable (mode collapse)
DCGAN (2016)	G: transposed conv (fractionally strided) D: conv nets	Same minimax BCE	Stable training via convolutions; learns hierarchical features	Image generation, unsupervised representation learning	Output 64×64×3 after 5 epochs; G maps 100→4×4×1024→8×8×512→…→64×64
BGAN (2017)	Policy gradient generator; importance weights from D's decision boundary	Boundary-seeking policy gradient	Stable training with discrete data; outperforms vanilla GAN & proxy loss at same epochs	Discrete image / character-level text generation	5 G updates per 1 D update; avoids over-optimizing G
cGAN (2014)	G & D both receive extra label c	Conditional minimax BCE	Precise control over generated output (class/style)	Controlled image generation (e.g., specific digit)	Extra label improves quality; demonstrated on MNIST
AC-GAN (2016)	D outputs $(P(real), P(class))$ with auxiliary classifier	BCE + auxiliary classifier loss	High-quality class-conditional generation; scalable to many classes	Large-scale conditional synthesis (ImageNet)	100 models cover 1000 classes; each model handles 10 classes
SRGAN (2017)	G: deep CNN + transposed conv; D: CNN	Perceptual loss (VGG feature + adversarial)	Photo-realistic textures instead of blur; up to 4× up-scaling	Super-resolution (90×90 → 360×360)	Trains on 64×64 → 256×256 patches; avoids MSE blur
StackGAN (2017)	Two-stage: Stage-I G (low-res sketch) + Stage-II G (refinement)	Adversarial + text embedding	Decomposes hard problem; refines details consistently with text	Text-to-image (256×256)	Stage-I: rough shape/colour; Stage-II: high-res details; tested on CUB, Oxford-102, COCO
Disco GAN (2017)	Cross-domain mapping with cycle-consistency	Adversarial + cycle-consistency loss	Discovers domain relations without paired labels	Unpaired image style transfer / domain adaptation	No extra labels required
Flow-based GAN (2018)	G: invertible flow model; D: standard	Negative log-likelihood (MLE) + adversarial	Exact density evaluation + high sample quality	Generation with explicit likelihood evaluation	Bridges explicit (VAE, flow) and implicit (GAN) models
InfoGAN (2016)	G: standard; D shares layers with classification head	Maximise mutual information $I(c; G(z, c))$	Unsupervised disentanglement of interpretable factors	Discovering latent features (pose, width, lighting) on unlabeled data	Reveals meaningful latent codes without supervision

Conclusion and Comparison

GAN Variant	Architecture (G / D)	Loss / Objective	Key Advantage	Primary Application	Key Details & Scale
StyleGAN-XL (2022)	G: scale-separated conv blocks, shared backbone, domain-specific heads, stochastic feature modulation	Standard adversarial + multi-domain loss	State-of-the-art high-res ($> 1024^2$) synthesis; reduced aliasing; excellent generalization across diverse domains, even data-scarce ones	Multi-domain high-resolution image generation (natural, scientific)	Extends StyleGAN3; domain heads; excels in scientific imaging
GigaGAN (2023)	G: spectrally normalized transformer with Cross-Layer Attention Modulation (CLAM); D: standard	Text-conditional adversarial loss	10B-parameter GAN competitive with diffusion models; 10-20× faster sampling; coherent composition & fine textures	Large-scale text-to-image generation	10B parameters; trained on large multimodal datasets
Mask-Guided GAN (MGGAN) (2023)	G: takes image + semantic mask; D: standard	Adversarial + Mask Consistency Loss	Controllable editing with structural guarantees; reduces mode collapse via spatial attention	Facial attribute editing, scene rearrangement, medical image manipulation, partial in-painting	Mask supervision ensures structure integrity
Single-Image GAN (SinGAN++) (2024)	Pyramid of generators per scale with cross-scale residual links	Multi-scale adversarial + Progressive Consistency Regularization	Learns from a single image; supports editing (super-resolution, harmonization, animation); sharper textures	Extremely low-data synthesis, single-image editing	Improves SinGAN; progressive consistency avoids overfitting
U-GAN (2024)	G: U-Net-style encoder-decoder with skip connections & Adaptive Skip Modulation; D: standard	Adversarial loss (with possible reconstruction)	High spatial precision; stable training at high resolutions; pixel-accurate outputs	Medical imaging, satellite imagery, image restoration	U-Net generator; latent vector controls each resolution level independently